第 2 周：凸优化与 KKT 条件

从 NP-Hard 到多项式时间：凸性的魔力

目录

1 回顾：从复杂度到结构 **4**

2 凸集与凸函数 **5**
2.1 凸集的定义 5
2.2 凸函数的定义 6

3 凸性判定 **7**
3.1 一阶条件 7
3.2 二阶条件 8

4 凸的礼物：局部即全局 **10**

5 梯度下降的收敛性 ★ **11**

6 近端梯度法 ★ **15**

7 凸优化的标准形式 **21**
7.1 线性规划 (LP) 的例子 23
7.2 二次规划 (QP) 的例子 27
7.3 二阶锥规划 (SOCP) 的例子 29
7.4 半定规划 (SDP) 的例子 30

8 Lagrangian 与对偶函数 **35**

9 弱对偶与强对偶 **36**

11 对偶的经济学解释 **41**

12 机器学习视角：凸优化的现代应用 **44**

13 MDP 与线性规划 ★ **47**
13.1 具体例子：机器维护问题 49
13.2 Bellman 方程 50
13.3 MDP 的线性规划形式 51
13.4 LP 对偶与占用度量 54

14 GPU 改变现代优化 **56**

15 实验 **58**

15.1 实验 2a：CVXPY 求解 SVM 对偶 58

15.2 实验 2b：GPU vs CPU 凸优化 60

16 本周总结 **63**

17 思考题 **64**

18 附录：求解 MDP 的三条路 ★ **65**

本讲导读

内容层次

核心内容（课堂必讲，约 90 分钟）：

- 凸集、凸函数的定义与判定（一阶/二阶条件）
- 凸优化的标准形式，LP/QP 例子
- Lagrangian、对偶函数、弱对偶
- KKT 条件（四个条件 + 直觉）
- 实验 2a/2b

重要证明（标注 ★，选讲或自学）：

- 一阶条件、二阶条件的证明
- KKT 条件的推导
- Logistic 回归的凸性证明
- **梯度下降收敛性证明**：凸 L-光滑函数 $O(1/T)$ 收敛率；强凸函数线性收敛率

进阶内容（标注 ★，适合优秀学生）：

- MDP 与线性规划：序贯决策问题的优化视角
- Bellman 方程 $\to$ LP 不等式
- LP 对偶 $\leftrightarrow$ 值函数 vs 占用度量

扩展内容（自学材料）：

- SOCP、SDP 的详细例子
- 机器学习视角的深入讨论
- 影子价格的完整推导

数学记号

记号	含义
$\nabla f(x)$	梯度（列向量）
$\nabla^2 f(x)$	Hessian 矩阵
$A \succeq 0$	A 半正定（所有特征值 ≥ 0）
$A \succ 0$	A 正定（所有特征值 > 0）
$\langle A, B \rangle$	矩阵内积 $= \mathrm{tr}(A^T B)$
p^*	原问题最优值
d^*	对偶问题最优值
λ, ν	Lagrange 乘子

1 回顾：从复杂度到结构

第 1 周的核心结论

问题的难度层级：

P（高效）	NP	NP-Hard	不可判定
排序、最短路	SAT、TSP	一般优化	停机问题

→ 难度

关键问题：一般优化是 NP-Hard，我们怎么办？

从 NP-Hard 逃脱的方法

方法 1：接受近似解（启发式算法）

方法 2：利用问题的**特殊结构**

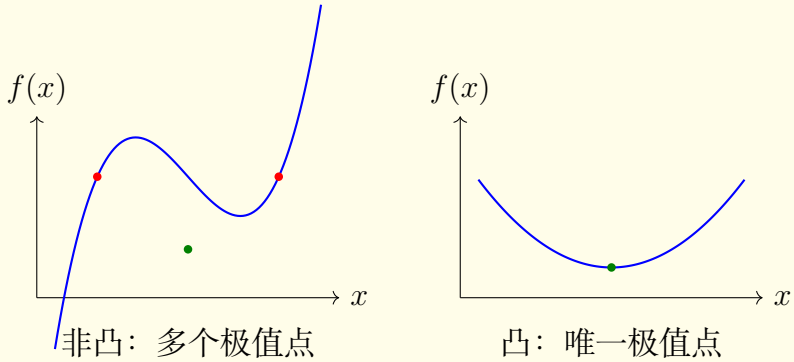
结构	复杂度	例子
无结构（一般）	NP-Hard	一般非线性优化
凸	**P**	**凸优化（本周重点）**
线性	P	线性规划
图结构	取决于具体问题	最短路、最小生成树

本周主题：**凸性**是最重要的“好结构”

直觉：凸性为什么重要？

优化的终极梦想：给我一个函数，我想找它的最小值

噩梦：函数有很多局部最小值，你找到的可能不是全局最优



非凸：多个极值点　　凸：唯一极值点

凸的礼物：局部最优 = 全局最优！

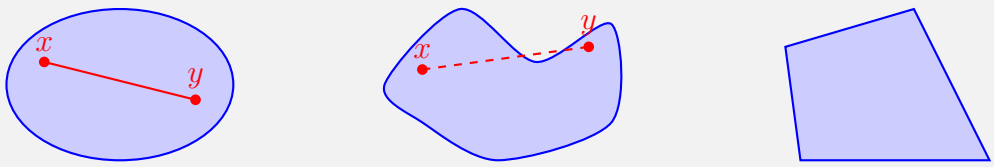
2 凸集与凸函数

2.1 凸集的定义

凸集：几何定义

定义 2.1 (凸集)**.** 集合 $C \subseteq \mathbb{R}^n$ 是**凸集**，当且仅当：
对任意 $x, y \in C$ 和任意 $\theta \in [0,1]$，有 $\theta x + (1-\theta)y \in C$

直觉：集合内任意两点的连线段都完全在集合内



凸集 ✓　　非凸集 ×　　多面体（凸）

常见的凸集

1. **超平面**：$\{x : a^T x = b\}$
2. **半空间**：$\{x : a^T x \le b\}$
3. **多面体**（半空间的交集）：$\{x : Ax \le b\}$
4. **球**：$\{x : \|x - x_0\| \le r\}$
5. **椭球**：$\{x : (x-x_0)^T P^{-1}(x-x_0) \le 1\}$，$P \succ 0$
6. **锥**：$\{x : \|x\| \le t\}$（二阶锥）

7. 半正定锥：$\{X \in \mathbb{R}^{n\times n} : X = X^T, X \succeq 0\}$
关键性质：凸集的交集仍是凸集

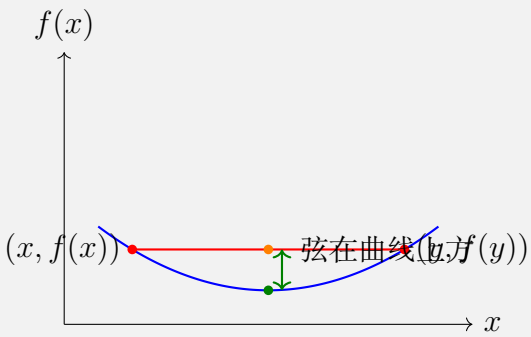
2.2 凸函数的定义

凸函数：几何定义

定义 2.2 (凸函数)**.** 函数 $f : \mathbb{R}^n \to \mathbb{R}$ 是**凸函数**，当且仅当：
对任意 x, y 和任意 $\theta \in [0, 1]$：

$$f(\theta x + (1-\theta)y) \le \theta f(x) + (1-\theta) f(y)$$

直觉：函数图像上任意两点连线在函数图像的**上方**



Jensen 不等式

定理 2.3 (Jensen 不等式)**.** 如果 f 是凸函数，$x_1, \ldots, x_k$ 是任意点，$\theta_1, \ldots, \theta_k \ge 0$ 且 $\sum_i \theta_i = 1$，则：

$$f\left(\sum_{i=1}^{k} \theta_i x_i\right) \le \sum_{i=1}^{k} \theta_i f(x_i)$$

直觉："加权平均后求函数值" $\le$ "函数值的加权平均"
特例（期望形式）：如果 X 是随机变量，f 是凸函数：

$$f(\mathbb{E}[X]) \le \mathbb{E}[f(X)]$$

应用：

- $f(x) = x^2$ 凸 $\Rightarrow (\mathbb{E}[X])^2 \le \mathbb{E}[X^2]$，即 $\mathrm{Var}(X) \ge 0$
- $f(x) = -\log x$ 凸 $\Rightarrow -\log(\mathbb{E}[X]) \le \mathbb{E}[-\log X]$（AM-GM 不等式）

常见的凸函数

一元凸函数：

- $x^2, |x|, e^x, x\log x \quad (x>0)$，$-\log x \quad (x>0)$

多元凸函数：

- 仿射函数：a^Tx+b（既凸又凹）
- 范数：$\|x\|_1, \|x\|_2, \|x\|_\infty$
- 二次函数：x^TPx，当 $P \succeq 0$
- Log-sum-exp：$\log(e^{x_1}+\cdots+e^{x_n})$
- 最大值函数：$\max\{x_1,\ldots,x_n\}$

保持凸性的操作：

- 非负加权和：$\sum_i \alpha_i f_i$，$\alpha_i \geq 0$
- 复合仿射：$f(Ax+b)$
- 逐点最大：$\max\{f_1(x), f_2(x)\}$

3 凸性判定

如何判断函数是否凸？

方法 1：直接验证定义（通常很难）
方法 2：一阶条件（需要可微）
方法 3：二阶条件（需要二次可微）
方法 4：证明它是凸函数的组合

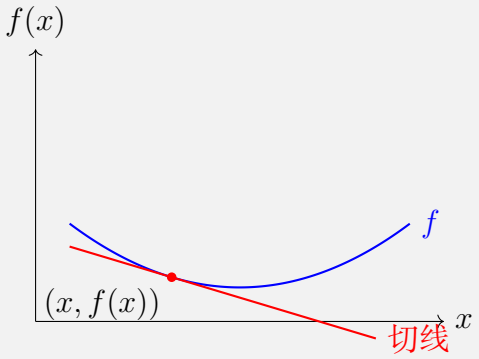
3.1 一阶条件

一阶条件：切线在下方

定理 3.1 (凸性的一阶条件)**.** 设 f 可微。f 是凸函数当且仅当对所有 x, y：

$$f(y) \geq f(x) + \nabla f(x)^T(y-x)$$

几何意义：函数图像始终在其切平面的**上方**



推论：如果 $\nabla f(x^*) = 0$，则 x^* 是**全局最小值**！

一阶条件的证明 ★

证明（$\Rightarrow$）：设 f 凸，要证 $f(y) \geq f(x) + \nabla f(x)^T(y-x)$
由凸性定义，对 $\theta \in (0,1)$：

$$f(x + \theta(y-x)) \leq (1-\theta)f(x) + \theta f(y)$$

移项：

$$f(x + \theta(y-x)) - f(x) \leq \theta(f(y) - f(x))$$

除以 θ：

$$\frac{f(x + \theta(y-x)) - f(x)}{\theta} \leq f(y) - f(x)$$

令 $\theta \to 0^+$，左边趋于方向导数 $\nabla f(x)^T(y-x)$：

$$\nabla f(x)^T(y-x) \leq f(y) - f(x)$$

即 $f(y) \geq f(x) + \nabla f(x)^T(y-x)$ □

3.2 二阶条件

二阶条件：Hessian 半正定

定理 3.2 (凸性的二阶条件). *设 f 二次可微。f 是凸函数当且仅当对所有 x：*

$$\nabla^2 f(x) \succeq 0$$

即 Hessian 矩阵处处半正定。

回忆：矩阵 H 半正定 $\Leftrightarrow$ 所有特征值 $\geq 0 \Leftrightarrow v^T H v \geq 0$ 对所有 v
直觉：二阶导数处处非负 $\Rightarrow$ 函数“向上弯曲”

二阶条件的证明 ★

证明（$\Rightarrow$）：设 f 凸，要证 $\nabla^2 f(x) \succeq 0$
由一阶条件，对任意 y：

$$f(y) \geq f(x) + \nabla f(x)^T (y - x)$$

同时，交换 x, y 的角色：

$$f(x) \geq f(y) + \nabla f(y)^T (x - y)$$

两式相加：

$$0 \geq (\nabla f(x) - \nabla f(y))^T (y - x) = -(\nabla f(y) - \nabla f(x))^T (y - x)$$

即 $(\nabla f(y) - \nabla f(x))^T (y - x) \geq 0$（梯度是**单调**的）
设 $y = x + tv$，$t > 0$：

$$(\nabla f(x + tv) - \nabla f(x))^T v \geq 0$$

除以 t，令 $t \to 0$：

$$v^T \nabla^2 f(x) v \geq 0$$

对所有 v 成立，故 $\nabla^2 f(x) \succeq 0$ □

例子：二次函数

函数：$f(x) = \frac{1}{2} x^T P x + q^T x + r$
梯度：$\nabla f(x) = Px + q$
Hessian：$\nabla^2 f(x) = P$
凸性：f 是凸函数 $\Leftrightarrow P \succeq 0$
强凸性：f 是强凸函数 $\Leftrightarrow P \succ 0$

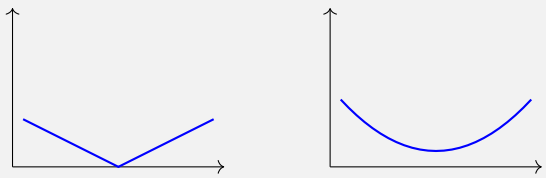
强凸性

定义 3.3 (强凸函数)**.** 函数 f 是 m-**强凸**的，如果对所有 x：

$$\nabla^2 f(x) \succeq mI, \quad m > 0$$

等价定义：$f(x) - \frac{m}{2}\|x\|^2$ 是凸函数
性质：

- 强凸函数有**唯**一的全局最小值
- 梯度下降收敛更快（线性收敛 vs 次线性收敛）



4 凸的礼物：局部即全局

凸优化的核心定理

定理 4.1 (局部最优 = 全局最优). 对于凸优化问题：

$$\min_x f(x) \quad s.t. \quad x \in C$$

其中 f 是凸函数，C 是凸集。
如果 x^* 是**局部最优解**，则 x^* 是**全局最优解**。

证明：反证法。假设存在 $y \in C$ 使得 $f(y) < f(x^*)$。
由凸性，对任意 $\theta \in (0,1)$：

$$f(\theta y + (1-\theta)x^*) \leq \theta f(y) + (1-\theta) f(x^*) < f(x^*)$$

当 $\theta \to 0$ 时，$\theta y + (1-\theta)x^* \to x^*$，矛盾于 x^* 是局部最优。

直觉：为什么这个定理如此重要？

非凸优化的噩梦：

- 梯度下降可能陷入局部最优
- 不知道找到的解有多好
- 可能需要多次随机重启

凸优化的美梦：

- 任何下降方法都能找到全局最优
- 有收敛性保证
- 一次运行就够

结论：证明问题是凸的，是优化研究的第一步！

5 梯度下降的收敛性 ★

基本假设与记号

设 $f:\mathbb{R}^n\to\mathbb{R}$ 是可微函数，$x^*=\arg\min_x f(x)$ 是全局最优解，$f^*=f(x^*)$。
梯度下降迭代（步长 $\alpha>0$）：

$$x_{t+1}=x_t-\alpha\nabla f(x_t)$$

本节使用以下两个核心假设：

1. **凸性**：f 是凸函数，即对所有 x,y：

$$f(y)\geq f(x)+\nabla f(x)^T(y-x)$$

2. **L-光滑性**（梯度 Lipschitz 连续）：存在常数 $L>0$，使得对所有 x,y：

$$\|\nabla f(x)-\nabla f(y)\|_2\leq L\|x-y\|_2$$

等价地，对所有 x,y：

$$f(y)\leq f(x)+\nabla f(x)^T(y-x)+\frac{L}{2}\|y-x\|^2\quad\text{（下降引理）}$$

直觉：L-光滑意味着函数不会”弯曲得太快”，梯度信息在一步之内仍然可靠。

关键引理

引理 5.1 (充分下降引理)**.** 设 f 是 L-光滑函数，步长 $\alpha=1/L$，则梯度下降每步保证：

$$f(x_{t+1})\leq f(x_t)-\frac{1}{2L}\|\nabla f(x_t)\|^2$$

证明：将 $y=x_{t+1}=x_t-\frac{1}{L}\nabla f(x_t)$ 代入下降引理：

$$f(x_{t+1})\leq f(x_t)+\nabla f(x_t)^T\left(-\frac{1}{L}\nabla f(x_t)\right)+\frac{L}{2}\left\|\frac{1}{L}\nabla f(x_t)\right\|^2=f(x_t)-\frac{1}{2L}\|\nabla f(x_t)\|^2 \qquad \square$$

推论：$f(x_t)$ 单调不增，即梯度下降在凸 L-光滑函数上**永远不会变差**。

主定理：$O(1/T)$ 收敛率

定理 5.2 (凸函数梯度下降收敛率)**.** 设 f 是凸函数且 L-光滑，步长 $\alpha=1/L$。则对任意 $T\geq 1$：

$$f(x_T)-f^*\leq\frac{L\,\|x_0-x^*\|^2}{2T}$$

证明：

第一步：建立距离递推关系。令 $r_t = \|x_t - x^*\|^2$，展开：

$$\begin{aligned} r_{t+1} &= \left\| x_t - \frac{1}{L}\nabla f(x_t) - x^* \right\|^2 \\ &= r_t - \frac{2}{L}\nabla f(x_t)^T(x_t - x^*) + \frac{1}{L^2}\|\nabla f(x_t)\|^2 \end{aligned}$$

第二步：利用凸性下界。由凸性一阶条件（取 $y = x^*$）：

$$f(x_t) - f^* \le \nabla f(x_t)^T(x_t - x^*)$$

令 $\delta_t = f(x_t) - f^*$，代入第一步：

$$r_{t+1} \le r_t - \frac{2}{L}\delta_t + \frac{1}{L^2}\|\nabla f(x_t)\|^2$$

第三步：利用充分下降引理消去梯度范数。由引理知 $\|\nabla f(x_t)\|^2 \le 2L(\delta_t - \delta_{t+1})$，故：

$$r_{t+1} \le r_t - \frac{2}{L}\delta_t + \frac{2}{L}(\delta_t - \delta_{t+1}) = r_t - \frac{2}{L}\delta_{t+1}$$

整理得：

$$\frac{2}{L}\delta_{t+1} \le r_t - r_{t+1}$$

第四步：对 $t = 0, 1, \ldots, T-1$ 求和（望远镜求和）：

$$\frac{2}{L}\sum_{t=1}^{T}\delta_t \le r_0 - r_T \le r_0 = \|x_0 - x^*\|^2$$

第五步：由充分下降引理知 δ_t 单调不增，故 $T \cdot \delta_T \le \sum_{t=1}^{T}\delta_t$，因此：

$$\frac{2T}{L}\delta_T \le \|x_0 - x^*\|^2 \implies \boxed{f(x_T) - f^* \le \frac{L\,\|x_0 - x^*\|^2}{2T}} \qquad \square$$

核心结论：$O(1/T)$ 收敛率的含义

- 要使误差 $\le \varepsilon$，需要迭代次数 $T = O(L\|x_0 - x^*\|^2/\varepsilon)$，即 $O(1/\varepsilon)$ 步。
- 每步计算量为 $O(n)$（只算一次梯度），总计算量 $O(n/\varepsilon)$。
- 这被称为**次线性收敛**（sublinear convergence）：误差以 $1/T$ 的速度衰减。
- 此速率对一般凸函数是**最优的**（信息论下界），即无法在相同假设下做得更好。

强化：强凸函数的线性收敛率 ★

若进一步假设 f 是 **μ-强凸**（$\mu > 0$），即对所有 x, y：

$$f(y) \geq f(x) + \nabla f(x)^T(y-x) + \frac{\mu}{2}\|y-x\|^2$$

定理 5.3 (强凸函数线性收敛率). 设 f 是 μ-强凸且 L-光滑，步长 $\alpha = 1/L$，则：

$$\|x_T - x^*\|^2 \leq \left(1 - \frac{\mu}{L}\right)^T \|x_0 - x^*\|^2$$

证明：
由强凸性（令 $y = x^*$）：

$$f(x_t) - f^* \geq \frac{\mu}{2}\|x_t - x^*\|^2 = \frac{\mu}{2} r_t$$

由第三步的结论 $r_{t+1} \leq r_t - \frac{2}{L}\delta_{t+1}$，利用充分下降 $\delta_{t+1} \geq \frac{\mu}{2} r_{t+1}$：
实际上，更直接地，从第一步和强凸性下界出发：

$$r_{t+1} \leq r_t - \frac{2}{L}\delta_t + \frac{1}{L^2}\|\nabla f(x_t)\|^2$$

由强凸性推出 $\|\nabla f(x_t)\|^2 \geq 2\mu\delta_t$（证明略），结合 $\frac{1}{L^2}\|\nabla f(x_t)\|^2 \leq \frac{2}{L}\delta_t - \frac{2}{L}\delta_{t+1}$，
更简洁地，利用强凸性的等价形式（强单调性）直接得：

$$\begin{aligned}
r_{t+1} &= \left\| x_t - \frac{1}{L}\nabla f(x_t) - x^* \right\|^2 \\
&= r_t - \frac{2}{L}\underbrace{\nabla f(x_t)^T(x_t - x^*)}_{\geq \frac{\mu}{2} r_t + \frac{1}{2L}\|\nabla f(x_t)\|^2} + \frac{1}{L^2}\|\nabla f(x_t)\|^2 \\
&\leq r_t - \frac{\mu}{L} r_t - \frac{1}{L^2}\|\nabla f(x_t)\|^2 + \frac{1}{L^2}\|\nabla f(x_t)\|^2 \\
&= \left(1 - \frac{\mu}{L}\right) r_t
\end{aligned}$$

（第二行利用了强凸性的等价不等式 $\nabla f(x)^T(x-y) \geq \frac{\mu}{2}\|x-y\|^2 + \frac{1}{2L}\|\nabla f(x) - \nabla f(y)\|^2$，取 $y = x^*$ 且 $\nabla f(x^*) = 0$。）
递归展开得：

$$\boxed{\|x_T - x^*\|^2 \leq \left(1 - \frac{\mu}{L}\right)^T \|x_0 - x^*\|^2} \qquad \square$$

核心结论：线性收敛率的含义

- 定义**条件数** $\kappa = L/\mu \geq 1$，收缩因子为 $1 - 1/\kappa$。
- 要使 $\|x_T - x^*\|^2 \leq \varepsilon$，需要 $T = O(\kappa \log(1/\varepsilon))$ 步，即 $O(\log(1/\varepsilon))$ 步——远优于 $O(1/\varepsilon)$！

- 这是**线性收敛**（linear convergence）：误差的对数随迭代线性下降，即”每次迭代误差减少一个固定百分比”。
- κ 越大（病态程度越高），收敛越慢；$\kappa = 1$ 时（即 $\mu = L$）一步到达最优。

假设	收敛率	步数达到精度 ε	名称
凸 + L-光滑	$O(1/T)$	$O(1/\varepsilon)$	次线性收敛
强凸 + L-光滑	$O((1-\mu/L)^T)$	$O(\kappa \log 1/\varepsilon)$	线性收敛

例 5.4 (具体例子：$f(x) = kx^2$ 的梯度下降). 取 $f(x) = kx^2$，$k > 0$，最优解显然是 $x^* = 0$，$f^* = 0$。

第一步：确定 L 和 μ

$f''(x) = 2k$（常数），故 f 同时是 $L = 2k$-光滑的和 $\mu = 2k$-强凸的，条件数 $\kappa = L/\mu = 1$。

第二步：写出迭代格式

$\nabla f(x) = 2kx$，梯度下降：

$$x_{t+1} = x_t - \alpha \cdot 2kx_t = (1 - 2k\alpha)\, x_t$$

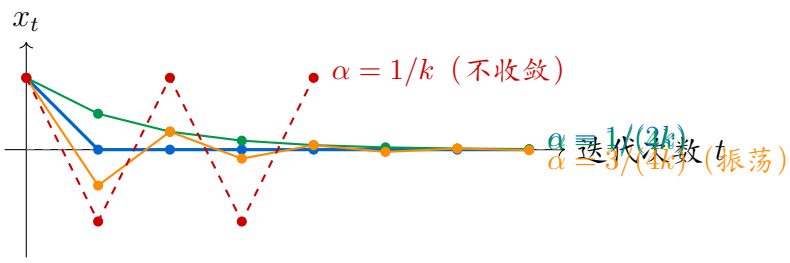
记收缩因子 $\rho = 1 - 2k\alpha$，递归展开得**闭合公式**：

$$\boxed{x_t = \rho^t\, x_0 = (1 - 2k\alpha)^t\, x_0}$$

第三步：分析不同步长的行为

步长 α	$\rho = 1 - 2k\alpha$	$\lvert\rho\rvert$	行为
$\frac{1}{2k}$（$= 1/L$，最优）	0	0	**一步收敛！** $x_1 = 0$
$\frac{1}{4k}$	$\frac{1}{2}$	0.5	单调收敛，每步误差减半
$\frac{3}{4k}$	$-\frac{1}{2}$	0.5	振荡收敛（正负交替，幅度减半）
$\frac{1}{k}$（$= 2/L$，临界）	-1	1	永远振荡，不收敛
$> \frac{1}{k}$	$\lvert\rho\rvert > 1$	> 1	发散（步长太大）

直觉图（$k = 1$，$x_0 = 1$，不同步长的轨迹）：



第四步：与一般理论对照

- 最优步长 $\alpha = 1/L = 1/(2k)$，此时 $\rho = 0$——$\kappa = 1$ 的情形理论预测 $(1-\mu/L)^T = 0^T = 0$，即一步精确到达，与计算一致。
- 步长约束 $0 < \alpha < 1/k = 2/L$ 与充分下降引理的要求 $\alpha \le 1/L$ 对应（$\alpha \le 1/L$ 保证单调下降，$1/L < \alpha < 2/L$ 振荡但仍收敛，$\alpha \ge 2/L$ 发散）。
- 误差公式 $|x_t - x^*| = |\rho|^t |x_0|$ 正是线性收敛 $O((1-\mu/L)^T)$ 的具体体现。

直觉：为什么凸性保证梯度下降找到最优解？

核心逻辑链：

1. **凸性 ⇒ 局部 = 全局**：不存在”陷阱”式局部最优。
2. **L-光滑 ⇒ 每步必下降**（充分下降引理）：梯度下降永远在改善。
3. **两者合并 ⇒ 收敛到全局最优**：既然每步都在改善，且改善终点就是全局最优，梯度下降必然收敛至全局最优。

对比非凸情形：满足下降引理，但可能收敛到局部最优——有下降保证，却无全局保证。

6 近端梯度法 ★

问题设置：光滑 + 不可微

许多实际优化问题的目标函数由两部分组成：

$$\min_x \quad \underbrace{f(x)}_{\text{光滑凸（有梯度）}} + \underbrace{g(x)}_{\text{不可微凸（无梯度）}}$$

典型的 g：

$g(x)$	名称	用途
$\lambda\|x\|_1$	ℓ_1 范数	稀疏回归（LASSO）、特征选择
$\mathbf{1}_C(x)$	指示函数	投影到凸集（等式 / 不等式约束）
$\lambda\|X\|_*$	核范数	矩阵补全（低秩约束）
$\lambda\|\nabla x\|_1$	Total Variation	图像去噪（保边缘）

为什么不能直接梯度下降？ g 不可微，∇g 不存在，无法直接写出迭代格式。

推导：从二次近似出发

普通梯度下降的本质：每步最小化 f 在当前点 $x^{(t)}$ 处的二次上界：

$$x^{(t+1)} = \arg\min_x \underbrace{f(x^{(t)}) + \nabla f(x^{(t)})^\top (x - x^{(t)}) + \frac{L}{2}\|x - x^{(t)}\|^2}_{f\text{ 的二次近似上界（}L\text{-光滑保证）}}$$

化简后就是 $x^{(t+1)} = x^{(t)} - \frac{1}{L}\nabla f(x^{(t)})$。

加入不可微项 g：保持 g 原样，只对 f 做二次近似：

$$\begin{aligned} x^{(t+1)} &= \arg\min_x \quad g(x) + f(x^{(t)}) + \nabla f(x^{(t)})^\top (x - x^{(t)}) + \frac{L}{2}\|x - x^{(t)}\|^2 \\ &= \arg\min_x \quad g(x) + \frac{L}{2}\left\| x - \underbrace{\left(x^{(t)} - \frac{1}{L}\nabla f(x^{(t)})\right)}_{z^{(t)}\text{：梯度步之后的点}} \right\|^2 \end{aligned}$$

（常数项 $f(x^{(t)}) - \frac{1}{2L}\|\nabla f(x^{(t)})\|^2$ 与 x 无关，已略去。）

分两步理解：

1. **梯度步**：对光滑部分 f 做一步梯度下降，得到 $z^{(t)} = x^{(t)} - \frac{1}{L}\nabla f(x^{(t)})$
2. **近端步**：把 $z^{(t)}$ ”投影”回让 g 小的区域，得到 $x^{(t+1)}$

近端算子的定义

定义 6.1 (近端算子)**.** *函数 g 的近端算子（proximal operator）*：

$$\mathrm{prox}_{\frac{1}{L}g}(z) = \arg\min_x \quad g(x) + \frac{L}{2}\|x - z\|^2$$

含义：找一个既让 g 小（第一项）、又离 z 不远（第二项）的点；$1/L$ 控制两者的权衡。

关键性质：

- 当 g 是凸函数时，近端算子有**唯一解**（严格凸的 $\|x - z\|^2$ 确保唯一性）
- 近端算子是**非扩张映射**：$\|\mathrm{prox}(z) - \mathrm{prox}(z')\| \le \|z - z'\|$
- 当 $g = 0$ 时，$\mathrm{prox}(z) = z$（退化为恒等映射）

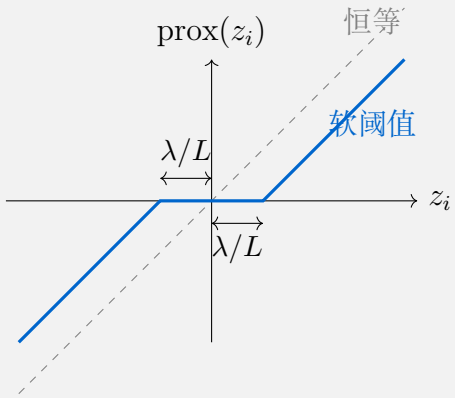
完整算法（近端梯度法，PGM）：

$$\boxed{z^{(t)} = x^{(t)} - \frac{1}{L}\nabla f(x^{(t)}), \qquad x^{(t+1)} = \mathrm{prox}_{\frac{1}{L}g}\big(z^{(t)}\big)}$$

三个最重要的近端算子

1. $g(x) = \lambda\|x\|_1$ **（LASSO / 稀疏约束）**
逐分量求解：对每个分量 $\min_{x_i} \lambda|x_i| + \frac{L}{2}(x_i - z_i)^2$，得**软阈值算子**：

$$\mathrm{prox}(z_i) = \mathrm{sign}(z_i) \cdot \max\left(|z_i| - \frac{\lambda}{L},\ 0\right)$$



绝对值 $< \lambda/L$ 的分量**精确置零**（稀疏性的来源！），其余向零收缩 λ/L。

2. $g(x) = \mathbf{1}_C(x)$ **（凸集指示函数）**
$\mathbf{1}_C(x) = 0$（若 $x \in C$）或 $+\infty$（否则）。近端算子退化为**投影**：

$$\mathrm{prox}(z) = \mathrm{Proj}_C(z) = \arg\min_{x \in C} \|x - z\|^2$$

近端梯度法退化为**投影梯度下降（PGD）**：先做梯度步，再投影回可行域。

3. $g(X) = \lambda\|X\|_*$ **（核范数，矩阵补全）**
设 $Z = U\Sigma V^\top$（SVD），近端算子为**奇异值软阈值（SVT）**：

$$\mathrm{prox}(Z) = U\ \mathrm{diag}\left(\max\left(\sigma_i - \frac{\lambda}{L},\ 0\right)\right) V^\top$$

对奇异值做软阈值——小奇异值置零，降低矩阵秩，推动低秩结构。

近端算子为什么”不用解优化”？

澄清一个常见误解：近端算子本身确实是个优化问题：

$$\mathrm{prox}_{\alpha g}(z) = \arg\min_x \quad g(x) + \frac{1}{2\alpha}\|x - z\|^2$$

并没有回避优化。说”不用解优化”的意思是：对常见的 g，这个问题有**闭合形式解**，直接套公式 $O(n)$ 算完，**无需迭代**。

$g(x)$	$\mathrm{prox}_{\alpha g}(z)$ 的闭合形式	计算量
$\lambda\|x\|_1$	软阈值，逐元素操作	$O(n)$
$\mathbf{1}_C(x)$，C 是半空间	一次内积 + 截断	$O(n)$
$\mathbf{1}_C(x)$，C 是球	归一化	$O(n)$
$\lambda\|x\|_2$（group lasso）	向量软阈值	$O(n)$
$\lambda\|X\|_*$	SVD + 奇异值软阈值	$O(n^3)$

ℓ_1 近端算子闭合解的推导

标量情形（$n=1$，$g(x)=\lambda|x|$）：

$$\min_x \quad \lambda|x| + \frac{1}{2\alpha}(x-z)^2$$

$\lambda|x|$ 在 $x=0$ 处不可微，用**次梯度最优性条件**（$0 \in \partial F(x)$）：

$$\frac{x-z}{\alpha} + \lambda \cdot \partial|x| \ni 0$$

分三种情况讨论：

1. **若** $x>0$：$\partial|x|=\{1\}$，条件变为 $\frac{x-z}{\alpha}+\lambda=0$，解得 $x=z-\alpha\lambda$，要求 $z>\alpha\lambda$。
2. **若** $x<0$：$\partial|x|=\{-1\}$，条件变为 $\frac{x-z}{\alpha}-\lambda=0$，解得 $x=z+\alpha\lambda$，要求 $z<-\alpha\lambda$。
3. **若** $x=0$：$\partial|x|=[-1,1]$，条件为 $\frac{-z}{\alpha}\in[-\lambda,\lambda]$，即 $|z|\le\alpha\lambda$。

三种情况合并，得**软阈值公式**（无需任何迭代）：

$$\mathrm{prox}_{\alpha\lambda|\cdot|}(z) = \mathrm{sign}(z)\cdot\max(|z|-\alpha\lambda,\ 0)$$

向量推广：$g(x)=\lambda\|x\|_1=\sum_i \lambda|x_i|$ 各分量独立，对每维用上式，$O(n)$ 完成。
为什么 ℓ_1 产生精确零而 ℓ_2 不能？
对 $g(x)=\lambda x^2$（ℓ_2），近端算子为 $\mathrm{prox}(z)=\frac{z}{1+2\alpha\lambda}$——对任意 $z\neq 0$ 输出均非零，只是缩小，永远无法置零。ℓ_1 的”死区”$|z|\le\alpha\lambda$ 使得该区间内的分量被**精确拉到零**，这是稀疏性的几何根源。

当近端算子没有闭合解时：ADMM

g 结构复杂时，近端算子本身也需要迭代，近端梯度法优势消失：

- $g(x)=\log(1+e^x)$（logistic 损失）：prox 无闭合解

- $g(x) = \|Ax\|_1$，A 非对角：各分量被 A 耦合，不可分离
- 一般凸集的指示函数（如 $X \succeq 0$ 且 $\mathcal{A}(X) = b$）：投影无公式

解法：ADMM（交替方向乘子法）——引入辅助变量把 f 和 g 拆开：

$$\min_{x,y} \quad f(x) + g(y) \quad \text{s.t.} \quad x = y$$

三步交替更新（$\rho > 0$ 为惩罚参数）：

$$\begin{aligned}
x^{(t+1)} &= \arg\min_x \quad f(x) + \frac{\rho}{2}\|x - y^{(t)} + u^{(t)}\|^2 \quad \text{（只含 } f\text{）} \\
y^{(t+1)} &= \operatorname{prox}_{\frac{1}{\rho}g}\big(x^{(t+1)} + u^{(t)}\big) \quad \text{（只含 } g\text{，用近端算子）} \\
u^{(t+1)} &= u^{(t)} + x^{(t+1)} - y^{(t+1)} \quad \text{（对偶变量更新，} O(n)\text{）}
\end{aligned}$$

只要 f 的子问题和 g 的近端算子各自易于求解，整体就高效。

具体例子：LASSO 的 ADMM 分解

原问题 $\min_x \|Ax - b\|^2 + \lambda\|x\|_1$，令 $y = x$：

- **x-子问题**：$\min_x \|Ax - b\|^2 + \frac{\rho}{2}\|x - y^{(t)} + u^{(t)}\|^2$

  这是**岭回归**，闭合解 $x = (2A^\top A + \rho I)^{-1}(2A^\top b + \rho(y^{(t)} - u^{(t)}))$。

  矩阵 $(2A^\top A + \rho I)$ **预先分解一次**（Cholesky，$O(n^3)$），之后每步只需回代 $O(n^2)$。

- **y-子问题**：$\min_y \lambda\|y\|_1 + \frac{\rho}{2}\|y - (x^{(t+1)} + u^{(t)})\|^2$

  这是 ℓ_1 的近端算子，直接**软阈值** $O(n)$：$y_i = \operatorname{sign}(v_i)\max(|v_i| - \lambda/\rho, 0)$，其中 $v = x^{(t+1)} + u^{(t)}$。

- **u-更新**：$u \leftarrow u + x - y$，$O(n)$。

每次迭代代价：矩阵向量乘 $O(mn)$（利用预分解）+ 软阈值 $O(n)$。与直接近端梯度法（每步也需 $O(mn)$ 计算 $A^\top(Ax - b)$）代价相当，但 ADMM 把”难的耦合”藏进了预分解，适合 A 结构特殊（稀疏、Toeplitz 等）的情形。

收敛率分析

定理 6.2 (近端梯度法收敛率)**.** *f 是 L-光滑凸函数，g 是凸函数，步长 $\alpha = 1/L$，则：*

$$f(x^{(t)}) + g(x^{(t)}) - (f^* + g^*) \le \frac{L\|x^{(0)} - x^*\|^2}{2t} = O\left(\frac{1}{t}\right)$$

与普通梯度下降（只有 f，$g = 0$）**完全相同的** $O(1/t)$ **收敛率**，但处理了不可微

项。

加速版（FISTA，Beck & Teboulle 2009）：加入动量（Nesterov 加速）：

$$y^{(t)} = x^{(t)} + \frac{t-1}{t+2}\big(x^{(t)} - x^{(t-1)}\big)$$

$$x^{(t+1)} = \operatorname{prox}_{\frac{1}{L}g}\left(y^{(t)} - \frac{1}{L}\nabla f(y^{(t)})\right)$$

收敛率提升到 $O(1/t^2)$，是一阶方法的理论最优（在凸 + L-光滑条件下）。

算法	收敛率	每步代价
梯度下降（$g=0$）	$O(1/t)$	$O(n)$ 梯度
近端梯度法（PGM）	$O(1/t)$	$O(n)$ 梯度 + prox
FISTA	$O(1/t^2)$	$O(n)$ 梯度 + prox（同 PGM）

动量带来 $O(1/t^2)$ 而每步代价与 PGM 完全相同——免费的加速！

三个具体应用

1. LASSO（稀疏回归）

$$\min_x \underbrace{\|Ax-b\|^2}_{f,\ \text{光滑}} + \underbrace{\lambda\|x\|_1}_{g,\ \text{不可微}}$$

$\nabla f(x) = 2A^\top(Ax-b)$，近端算子为软阈值。每步：梯度步 + 软阈值。

ℓ_1 惩罚把小系数压到**精确的零**（而不是趋近零），天然产生稀疏解。这是压缩感知、基因选择的核心算法。

2. 矩阵补全（Netflix 问题）

$$\min_X \underbrace{\sum_{(i,j)\in\Omega}(X_{ij}-M_{ij})^2}_{f} + \underbrace{\lambda\|X\|_*}_{g}$$

近端算子为奇异值软阈值（SVT）。每步：梯度步（仅在观测位置计算残差）+ SVT（压低矩阵秩）。

3. 图像去噪（Total Variation）

$$\min_x \underbrace{\|x-b\|^2}_{f} + \underbrace{\lambda\|\nabla x\|_1}_{g}$$

g 是图像梯度的 ℓ_1 范数，惩罚不连续性但容忍边缘跳变。近端算子有闭合形式（Chambolle 算法），能**保留边缘、去除噪声**。

直觉：近端梯度法的核心思想

近端梯度法的精髓在于**分而治之**：

- 光滑部分 f：用梯度（一阶信息）处理——廉价、可并行
- 不可微部分 g：用近端算子精确处理——利用 g 的特殊结构

只要能高效计算 prox_g，就能处理任意的 $f+g$ 形式。

近端算子 vs. 投影的关系：投影梯度下降是近端梯度法当 $g=\mathbf{1}_C$ 时的特例；核范数软阈值是矩阵版本的”投影”到低秩约束的松弛。三个例子本质上都在做同一件事：把”走了一步梯度”的中间点拉回到满足某种结构的地方。

7 凸优化的标准形式

直觉：既然梯度下降能解无约束凸优化，为什么还要搞有约束的？

问题：上一节证明了，对无约束凸优化 $\min_x f(x)$，梯度下降必然收敛到全局最优。那为什么不把所有问题都写成无约束形式，直接梯度下降了事？

原因一：现实问题本身就有约束

很多约束无法”消掉”，它们是问题的本质：

- **资源限制**：预算 ≤ 100 万、时间 ≤ 24 小时、库存 ≥ 0
- **物理约束**：功率 $\leq$ 额定值、速度 $\leq$ 光速
- **规则约束**：概率之和 $=1$、流量守恒等式
- **安全约束**：距离 $\geq$ 安全距离、应力 $\leq$ 材料极限

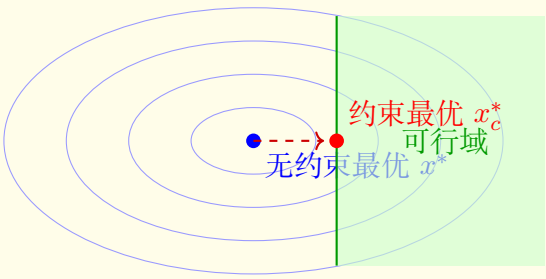
原因二：硬约束 $\neq$ 罚函数

一种”逃脱”约束的思路是把约束加到目标函数里（罚函数法）：

$$\min_x f(x)+\rho\cdot\max(0,\, g(x))$$

但这只是近似——罚函数法找到的解**一般不严格满足约束**，ρ 调多大都只是逼近。而有些场景（安全、合规、物理可行性）要求约束**精确成立**，差一点都不行。

原因三：约束改变了最优解的位置



无约束最优 x^* 在可行域**之外**，梯度下降" 直奔"x^* 根本不会停在可行域边界上，给出的解是不可行的。
结论：有约束凸优化 $\neq$ 无约束凸优化。我们需要专门的理论（Lagrangian、KKT 条件）来刻画约束最优解的结构，以及专门的算法（投影梯度下降、内点法、ADMM）来高效求解。

凸优化的标准形式

标准形式：

$$\begin{aligned} \min_x \quad & f(x) \\ \text{s.t.} \quad & g_i(x) \le 0, \quad i = 1, \dots, m \\ & h_j(x) = 0, \quad j = 1, \dots, p \end{aligned}$$

凸优化的要求：

- 目标函数 f 是**凸函数**
- 不等式约束 g_i 是**凸函数**
- 等式约束 h_j 是**仿射函数**（$h_j(x) = a_j^T x - b_j$）

注意：等式约束只能是仿射的！
原因：$h(x) = 0$ 等价于 $h(x) \le 0$ 且 $-h(x) \le 0$，要同时凸只能是仿射

常见的凸优化问题

1. 线性规划 (LP)：

$$\min_x c^T x \quad \text{s.t.} \quad Ax \le b, \quad Cx = d$$

2. 二次规划 (QP)：

$$\min_x \frac{1}{2} x^T P x + q^T x \quad \text{s.t.} \quad Ax \le b$$

（$P \succeq 0$）

3. 二阶锥规划 (SOCP)：

$$\min_x c^T x \quad \text{s.t.} \quad \|A_i x + b_i\|_2 \le c_i^T x + d_i$$

4. 半定规划 (SDP)：

$$\min_X \langle C, X \rangle \quad \text{s.t.} \quad \langle A_i, X \rangle = b_i, \quad X \succeq 0$$

包含关系：LP $\subset$ QP $\subset$ SOCP $\subset$ SDP $\subset$ 凸优化

7.1 线性规划 (LP) 的例子

LP 例子 1：生产计划

场景：工厂生产两种产品 A 和 B

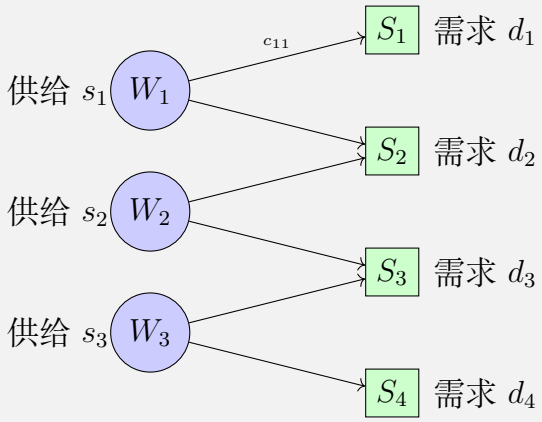
- 产品 A：利润 40 元，需要 2 小时机器时间，3 kg 原料
- 产品 B：利润 30 元，需要 1 小时机器时间，2 kg 原料
- 约束：每天最多 100 小时机器时间，120 kg 原料

建模：设 x_1, x_2 为两种产品的产量

$$\begin{aligned} \max_{x_1,x_2} \quad & 40x_1 + 30x_2 \\ \text{s.t.} \quad & 2x_1 + x_2 \le 100 \quad \text{（机器时间）} \\ & 3x_1 + 2x_2 \le 120 \quad \text{（原料）} \\ & x_1, x_2 \ge 0 \end{aligned}$$

LP 例子 2：运输问题

场景：3 个仓库向 4 个商店配送货物，最小化运输成本



建模：设 x_{ij} 为从仓库 i 到商店 j 的运输量

$$\begin{aligned} \min \quad & \sum_{i,j} c_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_j x_{ij} \le s_i \quad \text{（供给约束）} \\ & \sum_i x_{ij} \ge d_j \quad \text{（需求约束）} \\ & x_{ij} \ge 0 \end{aligned}$$

LP 例子 3：饮食问题

场景：以最低成本满足每日营养需求

食物	蛋白质 (g)	碳水 (g)	脂肪 (g)	价格 (元)
牛奶	8	12	8	3
鸡蛋	13	1	11	2
面包	4	52	1	2
每日需求	≥ 55	≥ 300	≥ 50	

建模：设 x_1, x_2, x_3 为三种食物的份数

$$\begin{aligned} \min \quad & 3x_1 + 2x_2 + 2x_3 \\ \text{s.t.} \quad & 8x_1 + 13x_2 + 4x_3 \geq 55 \\ & 12x_1 + x_2 + 52x_3 \geq 300 \\ & 8x_1 + 11x_2 + x_3 \geq 50 \\ & x_1, x_2, x_3 \geq 0 \end{aligned}$$

LP 的对偶问题推导

原始问题（标准不等式形式）：

$$p^* = \min_{x \geq 0} \; c^T x \quad \text{s.t.} \quad Ax \geq b$$

第一步：把约束改写成 ≤ 0 的标准形式

- 不等式约束 $Ax \geq b \Rightarrow b - Ax \leq 0$，引入乘子 $\lambda \geq 0$
- 非负约束 $x \geq 0 \Rightarrow -x \leq 0$，引入乘子 $\mu \geq 0$

第二步：写 Lagrangian

$$L(x, \lambda, \mu) = c^T x + \lambda^T (b - Ax) - \mu^T x = \underbrace{(c - A^T\lambda - \mu)^T}_{x \text{ 的系数}} x + \lambda^T b$$

第三步：对 x 取下确界（此时 x 无约束，非负已由 μ 处理）

$$g(\lambda, \mu) = \inf_{x \in \mathbb{R}^n} \left[(c - A^T\lambda - \mu)^T x + \lambda^T b \right] = \begin{cases} \lambda^T b & \text{若} c - A^T\lambda - \mu = 0 \\ -\infty & \text{否则} \end{cases}$$

仿射函数 $a^T x$ 在 $\mathbb{R}^n$ 上的下确界：若 $a \neq 0$，沿 $-a$ 方向趋于 $-\infty$；只有 $a = 0$ 时下确界有限。

第四步：写对偶问题

$\max_{\lambda,\mu}$ 自动排除 $g=-\infty$，等价于强制 $\mu = c - A^T\lambda$。
又 $\mu \geq 0$ 要求 $c - A^T\lambda \geq 0$，即 $A^T\lambda \leq c$。消去 μ：

$$\boxed{d^* = \max_{\lambda \geq 0} \; b^T\lambda \quad \text{s.t.} \quad A^T\lambda \leq c}$$

对称性一目了然：

	原始问题	对偶问题
变量	$x \geq 0$	$\lambda \geq 0$
目标	$\min\ c^T x$	$\max\ b^T\lambda$
约束	$Ax \geq b$	$A^T\lambda \leq c$
约束方向	$\geq$	$\leq$

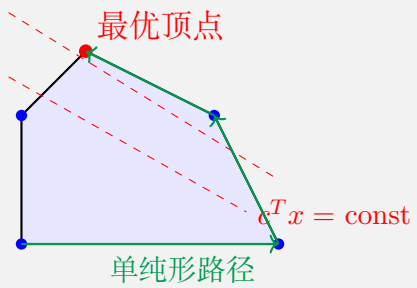
约束矩阵 A 变成 A^T；代价向量 c 变成对偶约束右端；原始约束右端 b 变成对偶目标系数。
对偶的对偶：对上面的对偶问题再做一次对偶，将 max 改 min、$\leq$ 改 $\geq$、$A^T \to A$、互换 b, c——恰好还原到原始问题（LP 强对偶由 Slater 条件保证）。

线性规划的两大算法：单纯形法与内点法

问题规模：n 个变量，m 条约束（$m \geq n$，标准形式）。

单纯形法（Simplex Method）

几何直觉：LP 的最优解必然在可行多面体的**顶点**（极点）上。



算法框架：

1. **初始化**：找一个初始顶点（基可行解，BFS）。
2. **检验**：计算相邻顶点的目标值是否有改善（用**简约费用** $\bar{c}_j = c_j - c_B^T B^{-1} A_j$）。
3. **转轴**（Pivot）：若存在 $\bar{c}_j < 0$（最小化问题），沿该方向移动到相邻顶点（**换基**）。

4. **终止**：所有 $\bar{c}_j \geq 0$，当前顶点即最优。

每步操作：更新基矩阵 B，求解 B^{-1}——每步代价 $O(mn)$（利用上次迭代的 LU 分解热启动）。
复杂度分析：

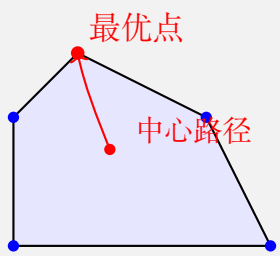
- **每步**：$O(mn)$
- **步数（最坏）**：$O\left(\binom{n}{m}\right) = 2^{\Omega(n)}$（Klee-Minty 例子），理论上指数级
- **步数（实践）**：通常 $O(m)$ 到 $O(3m)$ 步，实践中极快
- **总体**：理论 NP（最坏），实践近似 $O(m^2 n)$（多项式行为）

核心结论：单纯形法的悖论

最坏情况指数，但五十年来工程实践无敌。原因：实际问题的多面体"看起来"像球体（顶点数少、直径小），随机旋转规则下期望步数为 $O(m)$。

内点法（Interior Point Method / Barrier Method）

几何直觉：不走顶点，而是走**可行域内部**，沿中心路径（central path）趋向最优点。



核心思路：用对数障碍函数（log-barrier）将约束"软化"到目标里：

$$\min_x \; c^T x - \frac{1}{t} \sum_{i=1}^{m} \ln(-g_i(x))$$

随参数 $t \to \infty$，障碍项消失，解趋向原始最优解。
每个固定 t 下用 **Newton 法**求解（利用 Hessian 二阶信息），每步代价 $O(n^3)$（求解 $n \times n$ 线性方程组）或稀疏结构下更快。
复杂度分析：

- **每步（Newton 迭代）**：$O(n^3)$（稠密）或 $O(n^{1.5})$（稀疏/结构化）
- **总步数**：$O(\sqrt{m}\log(1/\varepsilon))$——这是**多项式时间**！

- **总体**：$O(n^3\sqrt{m}\log(1/\varepsilon))$，理论上优于单纯形

$O(\sqrt{m})$ 步的来源：每次 t 增大一个常数倍，中心路径的总”弯折程度”（自协调参数）为 $O(\sqrt{m})$。

对比总结：

	单纯形法	内点法	说明
每步代价	$O(mn)$	$O(n^3)$ 或 $O(n^{1.5})$	内点法每步更贵
步数（最坏）	指数	$O(\sqrt{m}\log 1/\varepsilon)$	内点法有理论保证
步数（实践）	$O(m)$	$O(30\text{–}100)$	内点法步数几乎与规模无关
适合规模	中小型，稀疏	中等密集型	各有适用场景
热启动	容易	困难	重复求解时单纯形更快
解的性质	顶点解（稀疏）	内部解（稠密）	影响后续处理

直觉：为什么 LP 有多项式算法但还在用指数的单纯形法？

- **理论**：内点法 $O(n^{3.5}\log 1/\varepsilon)$ 是多项式时间，1984 年 Karmarkar 算法证明 LP $\in$ P。
- **实践**：商业求解器（Gurobi、CPLEX）默认单纯形法，原因：
  1. 单纯形法的每步代价 $O(mn)$ 远小于内点法 $O(n^3)$
  2. 单纯形法解为顶点解（稀疏），便于热启动和整数规划
  3. 对大多数实际 LP，单纯形法 $O(m)$ 步的实践表现无敌
- **规模极大时**：一阶方法（投影梯度下降、ADMM）取代两者，每步 $O(mn)$ 但无需因子分解。

7.2 二次规划 (QP) 的例子

QP 例子 1：Markowitz 投资组合

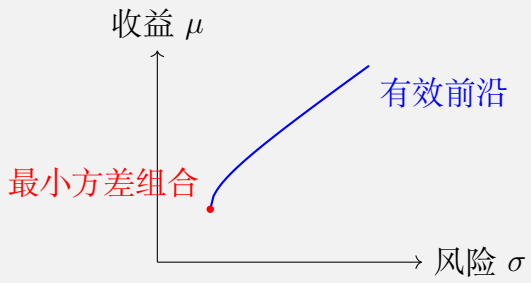
场景：投资 n 只股票，平衡收益与风险

- μ_i：股票 i 的预期收益率
- Σ：收益率的协方差矩阵（风险）
- x_i：投资在股票 i 的比例

建模：

$$\begin{aligned} \min_x \quad & \frac{1}{2}x^T\Sigma x \quad \text{（最小化风险）} \\ \text{s.t.} \quad & \mu^T x \geq r_{\min} \quad \text{（保证最低收益）} \\ & \mathbf{1}^T x = 1 \quad \text{（投资比例和为 1）} \\ & x \geq 0 \quad \text{（不允许卖空）} \end{aligned}$$

有效前沿：改变 $r_{\min}$，得到收益-风险权衡曲线



QP 例子 2：支持向量机 (SVM)

场景：二分类问题，找最大间隔超平面

原始问题：

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2}\|w\|^2 \\ \text{s.t.} \quad & y_i(w^T x_i + b) \geq 1, \quad \forall i \end{aligned}$$

这是一个 QP！目标函数是 w 的二次函数，约束是线性的。

软间隔 SVM（允许误分类）：

$$\begin{aligned} \min_{w,b,\xi} \quad & \frac{1}{2}\|w\|^2 + C\sum_i \xi_i \\ \text{s.t.} \quad & y_i(w^T x_i + b) \geq 1 - \xi_i \\ & \xi_i \geq 0 \end{aligned}$$

QP 例子 3：岭回归

场景：带正则化的线性回归

普通最小二乘：$\min_w \|Xw - y\|^2$

问题：当特征多、样本少时，过拟合严重

岭回归：

$$\min_w \|Xw - y\|^2 + \lambda\|w\|^2$$

展开：

$$\min_w w^T(X^TX+\lambda I)w - 2y^TXw + \text{const}$$

这是 QP！$P = 2(X^TX+\lambda I) \succ 0$（强凸）

闭式解：$w^* = (X^TX+\lambda I)^{-1}X^Ty$

7.3 二阶锥规划 (SOCP) 的例子

SOCP 例子 1：鲁棒线性规划

场景：参数有不确定性的 LP

普通 LP：$\min c^Tx$ s.t. $a_i^Tx \le b_i$

问题：如果 a_i 有误差怎么办?

假设 $a_i \in \mathcal{E}_i = \{\bar{a}_i + P_iu : \|u\| \le 1\}$（椭球不确定集）

鲁棒约束：对所有可能的 a_i 都满足

$$\max_{a_i\in\mathcal{E}_i} a_i^Tx \le b_i$$

等价于：

$$\bar{a}_i^Tx + \|P_i^Tx\| \le b_i$$

这是二阶锥约束！

SOCP 例子 2：设施选址

场景：在平面上选一个位置建设施，最小化到所有客户的总距离

客户位置：$(a_1,b_1),\dots,(a_m,b_m)$

建模：设设施位置为 (x,y)

$$\min_{x,y}\sum_{i=1}^m \sqrt{(x-a_i)^2+(y-b_i)^2}$$

引入辅助变量 t_i：

$$\begin{aligned}\min \quad & \sum_i t_i \\ \text{s.t.} \quad & \|(x-a_i, y-b_i)\|_2 \le t_i, \quad \forall i\end{aligned}$$

这是 SOCP！

SOCP 例子 3：机器人路径规划

场景：机器人从 A 到 B，避开圆形障碍物

障碍物：圆心 c_i，半径 r_i

约束：路径点 x_k 必须在所有障碍物外

$$\|x_k - c_i\| \geq r_i + \epsilon$$

问题：这是非凸的！（集合外部不是凸集）
解决方案：

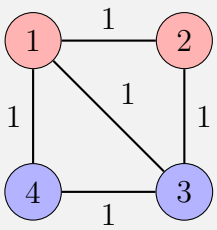
- 方法 1：凸松弛 + 序列凸规划 (SCP)
- 方法 2：整数规划（选择绕行方向）
- 方法 3：采样方法（RRT）

教训：不是所有几何问题都能直接凸化

7.4 半定规划 (SDP) 的例子

SDP 例子 1：MAX-CUT 松弛

MAX-CUT 问题：把图 $G = (V, E)$ 的顶点分成两组 S 和 $\bar{S}$，最大化跨组的边数（或加权边权之和）。
具体例子：4 个顶点、5 条边的图



切割 $\{1, 2\}$ vs $\{3, 4\}$
跨组边数 $= 4$

第一步：整数规划建模
令 $x_i \in \{-1, +1\}$：$+1$ 表示顶点 i 在 S，-1 表示在 $\bar{S}$。
边 (i, j) 被切割 $\Leftrightarrow x_i \neq x_j \Leftrightarrow x_i x_j = -1 \Leftrightarrow \frac{1 - x_i x_j}{2} = 1$。
原问题（NP-Hard 整数规划）：

$$\max_{x_i \in \{-1,+1\}} \sum_{(i,j) \in E} \frac{1 - x_i x_j}{2}$$

第二步：SDP 松弛
关键观察：$x_i x_j$（标量乘积）$\longrightarrow v_i^T v_j$（向量内积），把每个 $x_i \in \{-1, +1\}$ 替换为单位向量 $v_i \in \mathbb{R}^n$，$\|v_i\|_2 = 1$。
定义矩阵 $X_{ij} = v_i^T v_j$（Gram 矩阵）。Gram 矩阵天然满足：

- $X \succeq 0$（半正定）：因为 $u^T X u = \|\sum_i u_i v_i\|^2 \geq 0$

- $X_{ii} = v_i^T v_i = \|v_i\|^2 = 1$（对角元为 1）

SDP 松弛（以 X 为变量，$X \in \mathbb{R}^{4\times 4}$）：

$$\max_X \ \frac{1}{2} \sum_{(i,j)\in E} (1 - X_{ij}) \quad \text{s.t.} \quad X \succeq 0, \quad X_{ii} = 1 \ \ \forall i$$

本例目标：$\max \frac{1}{2}(5 - X_{12} - X_{23} - X_{34} - X_{14} - X_{13})$

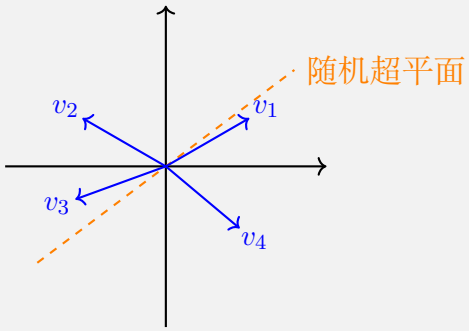
为什么这是松弛？原整数解 $x_i \in \{-1, +1\}$ 对应秩-1 的 X（$X = xx^T$），而 SDP 允许任意秩的半正定矩阵，可行域更大，最优值 $\geq$ 原整数问题最优值。

第三步：从 SDP 解恢复切割（Goemans-Williamson 随机舍入）

SDP 求解后得到 X^*，对其做 Cholesky 分解 $X^* = V^T V$，得到单位向量 $v_1, \ldots, v_4$。随机舍入：均匀随机选一个超平面法向量 $r \sim \mathcal{N}(0, I)$，令

$$x_i = \text{sign}(r^T v_i)$$

Goemans-Williamson 定理：该算法输出的期望切割值 $\geq 0.878 \times \text{OPT}$（OPT 为真实最优值），这是目前已知最好的多项式时间近似比。



超平面两侧分别染色 $\to$ 切割

SDP 例子 2：控制系统稳定性（Lyapunov LMI）

场景：线性时不变系统 $\dot{x}(t) = Ax(t)$，$x \in \mathbb{R}^n$，判断是否渐近稳定（即 $x(t) \to 0$）。

直接方法：看矩阵 A 的特征值是否全部有负实部——但这不能推广到有不确定性或时变的系统。SDP 方法更通用。

Lyapunov 稳定性理论

Lyapunov 函数：类比”能量”，若能找到一个正的”能量函数” $V(x)$ 沿轨迹严格递减，则系统稳定。

取 $V(x) = x^T P x$，$P \succ 0$（正定二次型）。沿轨迹求导：

$$\dot{V}(x) = \dot{x}^T P x + x^T P \dot{x} = x^T A^T P x + x^T P A x = x^T \underbrace{(A^T P + PA)}_{=:Q} x$$

要 V 沿轨迹严格递减，需要 $\dot{V}(x) < 0$ 对所有 $x \neq 0$，即 $Q \prec 0$（负定）。

定理 7.1 (Lyapunov 定理)**.** *线性系统 $\dot{x} = Ax$ 渐近稳定 $\iff$ 存在 $P \succ 0$ 使得 $A^T P + PA \prec 0$*

建模为 SDP 可行性问题
变量：对称矩阵 $P \in \mathbb{R}^{n\times n}$（共 $n(n+1)/2$ 个自由度）
约束均为线性矩阵不等式（LMI）：

$$\text{Find } P \quad \text{s.t.} \quad P \succ 0, \quad A^T P + PA \prec 0$$

这直接就是 SDP 可行性问题（目标函数取常数 0 即可）。
具体例子：

$$A = \begin{pmatrix} -1 & 2 \\ 0 & -3 \end{pmatrix}, \qquad P = \begin{pmatrix} p_{11} & p_{12} \\ p_{12} & p_{22} \end{pmatrix}$$

展开 $Q = A^T P + PA$：

$$A^T P + PA = \begin{pmatrix} -1 & 0 \\ 2 & -3 \end{pmatrix} \begin{pmatrix} p_{11} & p_{12} \\ p_{12} & p_{22} \end{pmatrix} + \begin{pmatrix} p_{11} & p_{12} \\ p_{12} & p_{22} \end{pmatrix} \begin{pmatrix} -1 & 2 \\ 0 & -3 \end{pmatrix} = \begin{pmatrix} -2p_{11} & 2p_{11} - p_{12} - 3p_{12} \\ \cdot & 4p_{12} - 6p_{22} \end{pmatrix}$$

整理（利用对称性）：

$$Q = \begin{pmatrix} -2p_{11} & 2p_{11} - 4p_{12} \\ 2p_{11} - 4p_{12} & 4p_{12} - 6p_{22} \end{pmatrix}$$

SDP 求解：取 $P = I$（$p_{11} = p_{22} = 1$，$p_{12} = 0$）验证：

$$Q = \begin{pmatrix} -2 & 2 \\ 2 & -6 \end{pmatrix}$$

验证 $Q \prec 0$：特征值 $= -4 \pm \sqrt{8} \approx -1.17, -6.83$，均 < 0。✓
进阶：最优化 Lyapunov 函数
可以把可行性问题改为优化问题，例如最大化 P 最小特征值（增强稳定裕度）：

$$\max_{P,\gamma} \quad \gamma \quad \text{s.t.} \quad P \succeq \gamma I, \quad A^T P + PA \preceq -\gamma I$$

仍然是 SDP，但现在有了意义明确的优化目标。
推广：对含不确定性的系统 $\dot{x} = (A + \Delta)x$，$\|\Delta\| \leq \epsilon$，鲁棒稳定性条件也能写成 LMI——这是 SDP 在控制领域的核心优势。

SDP 例子 3：矩阵补全（核范数最小化）

场景：观察到矩阵 $M \in \mathbb{R}^{m\times n}$ 的部分元素（由已知下标集 Ω 索引），恢复完整矩阵。

具体例子：3 个用户对 4 部电影的评分（? 表示未观测）

$$M = \begin{pmatrix} 5 & ? & 4 & ? \\ ? & 3 & ? & 2 \\ 4 & ? & 5 & ? \end{pmatrix}, \quad \Omega = \{(1,1),(1,3),(2,2),(2,4),(3,1),(3,3)\}$$

第一步：低秩假设

为什么矩阵是低秩的？用户 $\approx$ 少数几个" 口味类型" 的线性组合，电影 $\approx$ 少数几个" 题材特征" 的线性组合，因此评分矩阵 $\approx$ 秩 $r \ll \min(m,n)$ 的矩阵。

秩-1 示例：若所有用户只按" 动作 vs 文艺" 分类：

$$M = \underbrace{\begin{pmatrix} 0.9 \\ 0.4 \\ 0.8 \end{pmatrix}}_{\text{用户偏好}} \underbrace{\begin{pmatrix} 0.9 & 0.6 & 0.8 & 0.5 \end{pmatrix}}_{\text{电影特征}}$$

第二步：为何直接最小化秩行不通？

最直接的建模：$\min_X \operatorname{rank}(X)$ s.t. $X_{ij} = M_{ij}, \forall (i,j) \in \Omega$

但 $\operatorname{rank}(X)$ 是**非凸函数**（组合优化，NP-Hard）。

第三步：核范数——秩的凸松弛

核范数（nuclear norm）：$\|X\|_* = \sum_k \sigma_k(X)$，其中 σ_k 是奇异值。

类比：

量	非凸原版	凸松弛
向量稀疏度	$\|x\|_0$（非零元个数）	$\|x\|_1$（L1 范数）
矩阵秩	$\operatorname{rank}(X)$	$\|X\|_*$（核范数）

直觉：$\operatorname{rank}(X)$ = 非零奇异值的个数（L0" 范数"），$\|X\|_*$ = 奇异值之和（L1 范数），最小化核范数会促使尽量多的奇异值为零，即低秩。

核范数最小化问题：

$$\min_X \|X\|_* \quad \text{s.t.} \quad X_{ij} = M_{ij}, \quad \forall (i,j) \in \Omega$$

第四步：转化为 SDP

核范数的 SDP 等价表示：

$$\|X\|_* = \min_{W_1,W_2} \frac{1}{2}\big(\operatorname{tr}(W_1) + \operatorname{tr}(W_2)\big) \quad \text{s.t.} \quad \begin{pmatrix} W_1 & X \\ X^T & W_2 \end{pmatrix} \succeq 0$$

推导直觉：由 Schur 补，$\begin{pmatrix} W_1 & X \\ X^T & W_2 \end{pmatrix} \succeq 0$ 等价于 $W_1 \succeq XW_2^{-1}X^T$，迫使 W_1 的迹覆盖 X 的"能量"（奇异值），取最小值时恰好等于 $\|X\|_*$。
完整 SDP（变量：$X \in \mathbb{R}^{3\times 4}$，$W_1 \in \mathbb{R}^{3\times 3}$，$W_2 \in \mathbb{R}^{4\times 4}$）：

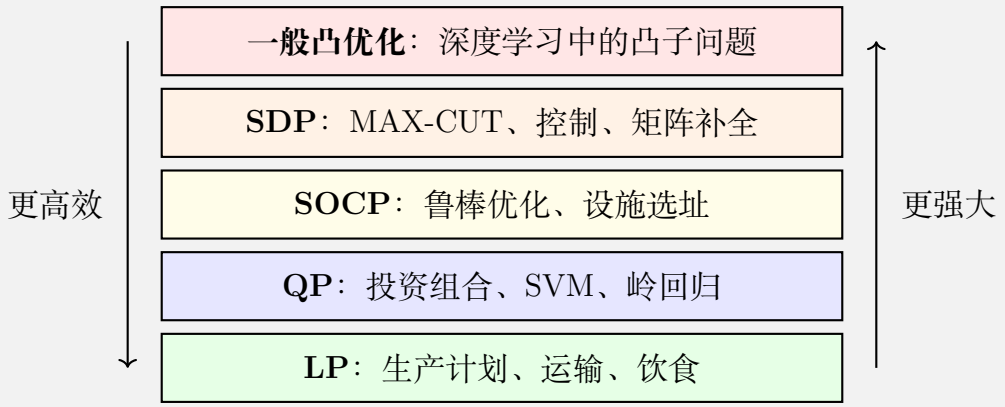
$$\begin{aligned} \min_{X,W_1,W_2} \quad & \frac{1}{2}\big(\mathrm{tr}(W_1) + \mathrm{tr}(W_2)\big) \\ \text{s.t.} \quad & \begin{pmatrix} W_1 & X \\ X^T & W_2 \end{pmatrix} \succeq 0 \\ & X_{11} = 5,\ X_{13} = 4,\ X_{22} = 3,\ X_{24} = 2,\ X_{31} = 4,\ X_{33} = 5 \end{aligned}$$

第五步：解读结果
SDP 解出后，X^* 即为补全矩阵（满足已知约束的最低秩近似）：

$$\hat{M} \approx \begin{pmatrix} 5 & 3.0 & 4 & 2.4 \\ 3.0 & 3 & 2.4 & 2 \\ 4 & 2.4 & 5 & 1.6 \end{pmatrix}$$

理论保证（Candès & Recht, 2009）：若真实矩阵秩为 r，且观测下标足够随机，则只需观测约 $O(rn\log n)$ 个元素，核范数最小化能以概率趋于 1 精确恢复完整矩阵。

凸优化问题层次总结



原则：用能解决问题的**最简单**的框架

8 Lagrangian 与对偶函数

Lagrangian 函数

考虑优化问题：

$$\begin{aligned} \min_x \quad & f(x) \\ \text{s.t.} \quad & g_i(x) \le 0, \quad i = 1, \dots, m \\ & h_j(x) = 0, \quad j = 1, \dots, p \end{aligned}$$

定义 8.1 (Lagrangian)**.** ***Lagrangian* 函数**：

$$L(x, \lambda, \nu) = f(x) + \sum_{i=1}^{m} \lambda_i g_i(x) + \sum_{j=1}^{p} \nu_j h_j(x)$$

其中 $\lambda_i \ge 0$ 是不等式约束的 ***Lagrange* 乘子**，ν_j 是等式约束的乘子。

向量形式：$L(x, \lambda, \nu) = f(x) + \lambda^T g(x) + \nu^T h(x)$

对偶函数

定义 8.2 (对偶函数)**.** ***Lagrange* 对偶函数**：

$$g(\lambda, \nu) = \inf_x L(x, \lambda, \nu) = \inf_x \left[f(x) + \lambda^T g(x) + \nu^T h(x) \right]$$

关键性质：对偶函数是**凹函数**（即使原问题非凸！）
原因：g 是关于 (λ, ν) 的仿射函数的逐点下确界

对偶函数的下界性质

定理 8.3 (弱对偶)**.** 对任意 $\lambda \ge 0$ 和任意 ν：

$$g(\lambda, \nu) \le p^*$$

其中 p^* 是原问题的最优值。

证明：设 $\tilde{x}$ 是可行解，则 $g_i(\tilde{x}) \le 0$，$h_j(\tilde{x}) = 0$。

$$\begin{aligned} g(\lambda, \nu) &= \inf_x L(x, \lambda, \nu) \\ &\le L(\tilde{x}, \lambda, \nu) \\ &= f(\tilde{x}) + \underbrace{\lambda^T g(\tilde{x})}_{\le 0} + \underbrace{\nu^T h(\tilde{x})}_{=0} \\ &\le f(\tilde{x}) \end{aligned}$$

对所有可行解都成立，故 $g(\lambda, \nu) \le p^*$。

9 弱对偶与强对偶

对偶问题

原问题（Primal）：

$$p^* = \min_x f(x) \quad \text{s.t.} \quad g(x) \le 0,\ h(x) = 0$$

对偶问题（Dual）：

$$d^* = \max_{\lambda \ge 0, \nu} g(\lambda, \nu) = \max_{\lambda \ge 0, \nu} \inf_x L(x, \lambda, \nu)$$

弱对偶（总是成立）：

$$d^* \le p^*$$

对偶间隙：$p^* - d^*$

强对偶

定义 9.1 (强对偶)**.** 如果 $d^* = p^*$，称**强对偶**成立。

什么时候强对偶成立？

定理 9.2 (Slater 条件)**.** 对于凸优化问题，如果存在**严格可行点** $\tilde{x}$：

$$g_i(\tilde{x}) < 0, \quad i = 1, \ldots, m; \quad h_j(\tilde{x}) = 0, \quad j = 1, \ldots, p$$

则强对偶成立。

直觉：约束不能“恰好”卡在边界上，要有“余量”

Slater 条件的证明 ★

目标：证明 $d^* = p^*$。由弱对偶 $d^* \le p^*$，只需证 $d^* \ge p^*$。
证明思路：构造两个不相交的凸集，用分离超平面定理”找出”对偶最优解，再用 Slater 条件排除退化情形。

第一步：构造两个不相交的凸集
定义”可达值域”（含松弛）：

$$\mathcal{A} = \left\{(u,\, v,\, t) \in \mathbb{R}^m \times \mathbb{R}^p \times \mathbb{R} \;\middle|\; \exists\, x \in \mathcal{D}:\ g(x) \preceq u,\ h(x) = v,\ f(x) \le t\right\}$$

定义”比最优值更好”的集合：

$$\mathcal{B} = \{(u, v, t) : u \preceq 0,\ v = 0,\ t < p^*\}$$

1. $\mathcal{A}$ 是凸集：若 $(u_1, v_1, t_1), (u_2, v_2, t_2) \in \mathcal{A}$，由 f、g_i 的凸性可验证其凸组合也在 $\mathcal{A}$ 内。
2. $\mathcal{B}$ 是凸集（线性约束定义的集合）。
3. $\mathcal{A} \cap \mathcal{B} = \emptyset$：若有 $(u, v, t) \in \mathcal{A} \cap \mathcal{B}$，则存在 x 满足 $g(x) \preceq u \preceq 0$、$h(x) = v = 0$、$f(x) \le t < p^*$——即 x 可行且 $f(x) < p^*$，与 p^* 是最优值矛盾。

第二步：分离超平面定理

$\mathcal{A}$ 与 $\mathcal{B}$ 均为凸集且不相交，由分离超平面定理，存在 $(\lambda, \nu, \mu) \neq 0$，使得

$$\lambda^T u + \nu^T v + \mu\, t \;\ge\; \lambda^T s + \mu\, r \quad \forall\, (u, v, t) \in \mathcal{A},\ (s, 0, r) \in \mathcal{B} \tag{$\star$}$$

从 $\mathcal{B}$ 的自由度推出符号约束：

- 令 $s_i \to -\infty$（$s \preceq 0$ 可任意负）：若某个 $\lambda_i < 0$，右边趋于 $+\infty$，违反 $(\star)$，故 $\lambda \ge 0$。
- 令 $r \to -\infty$（$r < p^*$ 可任意负）：同理推出 $\mu \ge 0$。

第三步：用 Slater 条件证 $\mu > 0$（关键步骤）

假设 $\mu = 0$。将严格可行点 $\tilde{x}$（满足 $g_i(\tilde{x}) < 0$，$h_j(\tilde{x}) = 0$）代入 $(\star)$ 左边，取 $s = 0$，r 任意：

$$\lambda^T g(\tilde{x}) + \nu^T \underbrace{h(\tilde{x})}_{=0} \;\ge\; \lambda^T \cdot 0 = 0 \quad\Longrightarrow\quad \lambda^T g(\tilde{x}) \ge 0$$

但 $\lambda \ge 0$ 且 $g_i(\tilde{x}) < 0$，故 $\lambda^T g(\tilde{x}) \le 0$。

两者合并得 $\lambda^T g(\tilde{x}) = 0$，又因为 $\lambda_i \ge 0$ 且 $g_i(\tilde{x}) < 0$，每个乘积 $\lambda_i g_i(\tilde{x}) \le 0$，因此每项必须为零，即 $\lambda = 0$。

此时 $\mu = 0$，$\lambda = 0$，但 $(\lambda, \nu, \mu) \neq 0$ 要求 $\nu \neq 0$。然而 h_j 是仿射函数（满射），$\nu^T h(x)$ 无界，无法被 $(\star)$ 左边一致下界——矛盾。

故 $\mu > 0$。

第四步：构造对偶最优解

对任意 $x \in \mathcal{D}$，$(u, v, t) = (g(x), h(x), f(x)) \in \mathcal{A}$，在 $(\star)$ 中令 $s = 0$，$r \to (p^*)^-$：

$$\lambda^T g(x) + \nu^T h(x) + \mu f(x) \;\ge\; \mu p^*$$

两边除以 $\mu > 0$，令 $\lambda^* = \lambda/\mu$，$\nu^* = \nu/\mu$：

$$f(x) + (\lambda^*)^T g(x) + (\nu^*)^T h(x) \;\ge\; p^* \quad \forall x \in \mathcal{D}$$

对 x 取下确界，注意左边正是 Lagrangian，得：

$$g(\lambda^*, \nu^*) = \inf_x L(x, \lambda^*, \nu^*) \ \geq \ p^*$$

故 $d^* = \sup_{\lambda \geq 0, \nu} g(\lambda, \nu) \geq g(\lambda^*, \nu^*) \geq p^*$。

第五步：结论

由弱对偶 $d^* \leq p^*$ 与上面的 $d^* \geq p^*$：

$$\boxed{d^* = p^*} \qquad \square$$

直觉：Slater 条件在证明中哪里起了作用？

证明的关键是排除退化情形 $\mu = 0$：

- 若 $\mu = 0$，超平面的法向量中”目标函数方向”消失，超平面完全由约束驱动，无法给出有意义的对偶变量。
- Slater 条件（严格可行点）保证了 $\lambda^T g(\tilde{x}) < 0$ 的可能性，迫使 $\lambda = 0$，从而与 $\nu \neq 0$ 产生矛盾。
- 本质上：严格可行点使得可行域”有内点”，排除了约束退化（线性无关约束失效）的情形。

反例（Boyd & Vandenberghe）：若 Slater 条件不满足，强对偶可能失败。考虑如下凸优化问题：

$$\min_{x_1, x_2} \ e^{-x_1} \quad \text{s.t.} \quad \frac{x_1^2}{x_2} \leq 0, \quad \mathcal{D} = \{(x_1, x_2) \mid x_2 > 0\}$$

目标函数 e^{-x_1} 和约束函数 x_1^2/x_2（x_1^2 的透视函数）均为凸函数，这确实是一个凸优化问题。

Slater 条件为何失效：要求存在 $x_2 > 0$ 使 $x_1^2/x_2 < 0$，但分子 $x_1^2 \geq 0$、分母 $x_2 > 0$，故 $x_1^2/x_2 \geq 0$ 恒成立，严格可行点不存在。

原问题最优值：约束 $x_1^2/x_2 \leq 0$ 与定义域 $x_2 > 0$ 联合要求 $x_1 = 0$，可行域为射线 $\{(0, x_2) \mid x_2 > 0\}$，目标值恒为 $e^0 = 1$，故 $p^* = 1$。

对偶问题最优值：拉格朗日函数 $L = e^{-x_1} + \lambda x_1^2/x_2$（$\lambda \geq 0$）。固定任意 $\lambda \geq 0$，令 $x_1 \to +\infty$ 且 $x_2 = x_1^3$，则

$$e^{-x_1} \to 0, \qquad \lambda \cdot \frac{x_1^2}{x_1^3} = \frac{\lambda}{x_1} \to 0$$

故 $g(\lambda) = \inf L = 0$，从而 $d^* = \max_{\lambda \geq 0} g(\lambda) = 0$。

对偶间隙：$p^* - d^* = 1 - 0 = 1 > 0$，强对偶失败。

1. **提供下界**：即使不能解原问题，对偶给出最优值的界
2. **有时对偶更容易**：对偶问题可能有更好的结构
3. **最优性条件**：强对偶 + KKT 条件 = 最优性的充要条件
4. **敏感性分析**：对偶变量告诉我们约束的 “价值”

KKT 条件的推导 ★

设强对偶成立：$p^* = d^*$

Step 1：写出对偶间隙为零的条件

设 x^* 是原问题最优解，(λ^*, ν^*) 是对偶问题最优解

$$
\begin{aligned}
0 &= p^* - d^* \\
&= f(x^*) - g(\lambda^*, \nu^*) \\
&= f(x^*) - \inf_x L(x, \lambda^*, \nu^*) \\
&\geq f(x^*) - L(x^*, \lambda^*, \nu^*) \quad \text{（下确界} \leq \text{任意值）} \\
&= f(x^*) - f(x^*) - \sum_i \lambda_i^* g_i(x^*) - \sum_j \nu_j^* h_j(x^*) \\
&= -\sum_i \underbrace{\lambda_i^*}_{\geq 0} \underbrace{g_i(x^*)}_{\leq 0} - \sum_j \nu_j^* \underbrace{h_j(x^*)}_{=0} \\
&\geq 0
\end{aligned}
$$

Step 2：由 $0 \geq 0$，所有不等式必须取等

- $\inf_x L(x, \lambda^*, \nu^*) = L(x^*, \lambda^*, \nu^*)$

  $\Rightarrow x^*$ 是 $L(\cdot, \lambda^*, \nu^*)$ 的最小值点

  $\Rightarrow \nabla_x L(x^*, \lambda^*, \nu^*) = 0$ **（稳定性）**

- $\sum_i \lambda_i^* g_i(x^*) = 0$，且每项 $\lambda_i^* g_i(x^*) \leq 0$

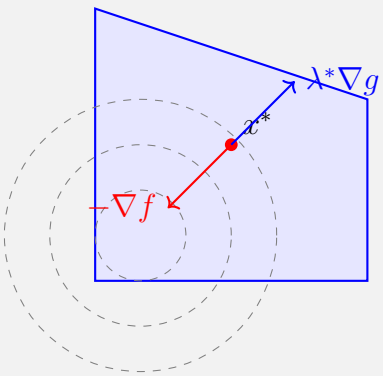
  $\Rightarrow \lambda_i^* g_i(x^*) = 0$ 对所有 i **（互补松弛）**

□

KKT 条件的直觉

稳定性：$\nabla_x L(x^*, \lambda^*, \nu^*) = 0$

在最优点，目标函数的梯度被约束的梯度 “平衡”



互补松弛：$\lambda_i^* g_i(x^*) = 0$

- 如果约束不紧（$g_i(x^*) < 0$），则 $\lambda_i^* = 0$（约束“不起作用”）
- 如果 $\lambda_i^* > 0$，则约束必须紧（$g_i(x^*) = 0$）

例子：带约束的二次规划

问题：

$$\min_x \frac{1}{2}x^2 \quad \text{s.t.} \quad x \geq 1$$

转换：$g(x) = 1 - x \leq 0$
Lagrangian：$L(x, \lambda) = \frac{1}{2}x^2 + \lambda(1 - x)$
KKT 条件：

1. 原始可行：$x \geq 1$
2. 对偶可行：$\lambda \geq 0$
3. 互补松弛：$\lambda(1 - x) = 0$
4. 稳定性：$x - \lambda = 0$

求解：由稳定性 $\lambda = x$。由互补松弛，要么 $\lambda = 0$（则 $x = 0$，违反原始可行），要么 $x = 1$。
故 $x^* = 1$，$\lambda^* = 1$。

11 对偶的经济学解释

影子价格：约束的边际价值

考虑带扰动的优化问题：

$$\begin{aligned} p^*(\epsilon) = \min_x \quad & f(x) \\ \text{s.t.} \quad & g_i(x) \le \epsilon_i, \quad i = 1, \dots, m \end{aligned}$$

当 $\epsilon = 0$ 时，就是原问题，最优值为 $p^*(0)$

问题：如果把第 i 个约束放松一点（$\epsilon_i > 0$），最优值会改善多少？

影子价格定理

定理 11.1 (影子价格)**.** 在最优解处，对偶变量 λ_i^* 满足：

$$\lambda_i^* = -\frac{\partial p^*}{\partial \epsilon_i}\bigg|_{\epsilon=0}$$

直觉：λ_i^* 告诉你 "放松约束 i 一个单位，目标函数能改善多少"

近似公式：当 ϵ 很小时

$$p^*(\epsilon) \approx p^*(0) - \sum_{i=1}^m \lambda_i^* \epsilon_i$$

具体例子：生产问题

问题：最小化成本，受资源约束

$$\begin{aligned} \min_x \quad & 2x_1 + 3x_2 \quad \text{（成本）} \\ \text{s.t.} \quad & x_1 + x_2 \ge 10 \quad \text{（产量需求）} \\ & x_1 \le 8 \quad \text{（机器 A 产能）} \\ & x_2 \le 6 \quad \text{（机器 B 产能）} \\ & x_1, x_2 \ge 0 \end{aligned}$$

转换为标准形式（$g_i(x) \le 0$）：

$$\begin{aligned} g_1(x) &= -x_1 - x_2 + 10 \le 0 \quad \text{（产量约束）} \\ g_2(x) &= x_1 - 8 \le 0 \quad \text{（机器 A）} \\ g_3(x) &= x_2 - 6 \le 0 \quad \text{（机器 B）} \end{aligned}$$

最优解：$x_1^* = 4$，$x_2^* = 6$，成本 $= 2(4) + 3(6) = 26$

影子价格的计算与解释

KKT 条件给出：$\lambda_1^* = 2$，$\lambda_2^* = 0$，$\lambda_3^* = 1$
解释：
约束 1（产量需求 ≥ 10）：$\lambda_1^* = 2$

- 如果只需要生产 9 件（放松 1 单位）
- 新问题：$g_1(x) = -x_1 - x_2 + 10 \leq 1$
- 成本减少约 $\lambda_1^* \times 1 = 2$ 元

约束 2（机器 A 产能 ≤ 8）：$\lambda_2^* = 0$

- 当前 $x_1^* = 4 < 8$，产能没用完
- 增加机器 A 产能没有价值！

约束 3（机器 B 产能 ≤ 6）：$\lambda_3^* = 1$

- 当前 $x_2^* = 6$，产能用尽
- 如果机器 B 产能增加到 7（放松 1 单位）
- 新问题：$g_3(x) = x_2 - 6 \leq 1$
- 成本减少约 $\lambda_3^* \times 1 = 1$ 元

验证：放松约束后重新求解

原问题：机器 B 产能 ≤ 6

- 最优解：$x_1^* = 4$，$x_2^* = 6$
- 成本：26

放松后：机器 B 产能 ≤ 7

- 新最优解：$x_1^* = 3$，$x_2^* = 7$
- 新成本：$2(3) + 3(7) = 27$?

等等，成本增加了？让我们重新思考...
注意：我们是在**最小化**成本，放松产能约束不一定降低成本！
正确理解：

- $\lambda_3^* = 1$ 意味着约束 “值 1 元”

- 如果**收紧**约束（机器 B 只能生产 5 件），成本会**增加**约 1 元
- 验证：$x_1 = 5, x_2 = 5$，成本 $= 2(5) + 3(5) = 25$...

重新审视：影子价格的符号取决于问题的设定方式！

影子价格的正确理解

关键公式：

$$p^*(\epsilon) \approx p^*(0) - \lambda^{*T}\epsilon$$

对于最小化问题：

- $\epsilon_i > 0$（放松约束）$\Rightarrow$ 可行域变大 $\Rightarrow p^*$ 可能变小
- $\lambda_i^* \geq 0$，所以 $-\lambda_i^*\epsilon_i \leq 0$

经济学直觉：

- $\lambda_i^* =$ 约束 i 的 "影子价格"
- $=$ 你愿意为 "放松约束一个单位" 支付的最高价格
- $=$ 约束的边际成本

实际应用：

- 生产计划：应该购买哪种资源？（看哪个 λ_i^* 最大）
- 电力市场：边际发电成本
- 物流优化：运力瓶颈在哪？

互补松弛的经济学意义

互补松弛：$\lambda_i^* g_i(x^*) = 0$
翻译成人话：

情况	数学	经济学
资源有剩余	$g_i(x^*) < 0$	供过于求
$\Rightarrow$ 价格为零	$\lambda_i^* = 0$	免费资源无价值
资源有价值	$\lambda_i^* > 0$	稀缺资源
$\Rightarrow$ 资源用尽	$g_i(x^*) = 0$	供不应求

直觉：只有**稀缺**的资源才有价格！

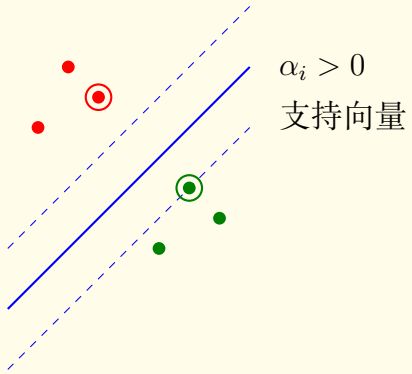
直觉：SVM 中的支持向量

SVM 对偶问题的变量 α_i 就是 Lagrange 乘子

互补松弛：$\alpha_i(1 - y_i(w^T x_i + b)) = 0$

- $\alpha_i = 0$：点 x_i 不在边界上（不是支持向量）
- $\alpha_i > 0$：点 x_i 恰好在边界上（是支持向量）

结论：只有**支持向量**对决策边界有贡献！



12 机器学习视角：凸优化的现代应用

机器学习 = 优化

几乎所有机器学习都可以写成：

$$\min_\theta \underbrace{\frac{1}{n}\sum_{i=1}^{n} \ell(f_\theta(x_i), y_i)}_{\text{经验风险}} + \underbrace{\lambda R(\theta)}_{\text{正则化}}$$

模型	损失函数 ℓ	正则化 R
线性回归	$(y - w^T x)^2$	$\|w\|_2^2$（岭回归）
Logistic 回归	$\log(1 + e^{-y \cdot w^T x})$	$\|w\|_2^2$ 或 $\|w\|_1$
SVM	$\max(0, 1 - y \cdot w^T x)$	$\|w\|_2^2$
神经网络	交叉熵	$\|W\|_F^2$（weight decay）

关键问题：哪些是凸的？哪些不是？

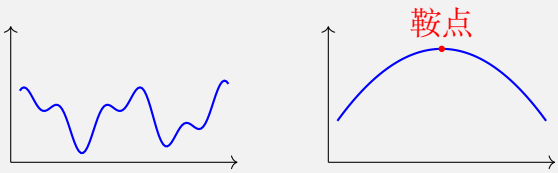
凸 vs 非凸机器学习

	凸	非凸
模型	线性回归、Logistic、SVM Lasso、岭回归	神经网络、矩阵分解 深度学习
优点	全局最优保证 理论完善	表达能力强 能学复杂模式
缺点	表达能力有限 只能学线性边界	可能陷入局部最优 理论不完善
优化方法	内点法、坐标下降 精确解或收敛保证	SGD、Adam 启发式，实践有效

为什么深度学习“不需要”凸优化？

传统观点：非凸 = 噩梦（局部最优、鞍点）
现代发现：深度网络的损失景观有特殊结构

1. **过参数化**：参数远多于数据，存在很多全局最优
2. **鞍点主导**：高维空间中，局部最优很少，大多是鞍点
3. **好的局部最优**：即使是局部最优，泛化性能也很好
4. **隐式正则化**：SGD 本身倾向于找“平坦”的最优解



低维：很多局部最优　　高维：鞍点主导

但是：凸优化的思想仍然重要！

凸优化思想在深度学习中的应用

1. 分层凸松弛
神经网络训练非凸，但每一层固定其他层后是凸的：

- 固定 W_2，优化 W_1：凸问题
- 交替优化：坐标下降的思想

2. 正则化 = 凸约束

L_2 正则化：$\min \text{Loss} + \lambda \|w\|_2^2$
等价于约束优化：$\min \text{Loss}$ s.t. $\|w\|_2 \le R$
对偶变量 $\lambda =$ 约束的影子价格！
3. 对偶在 Kernel 方法中的核心作用
SVM 对偶：

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

核技巧（Kernel Trick）完全依赖对偶形式！
4. KKT 条件 → 最优性检验
即使用 SGD 训练，也可以用 KKT 残差判断是否收敛

现代机器学习中的凸子问题

很多非凸问题包含凸子问题：

问题	凸子问题
神经网络训练	最后一层是凸的（固定特征）
EM 算法	M-step 通常是凸的
矩阵分解	固定一个因子，另一个是凸的
变分推断	ELBO 优化常是凸的
强化学习	Policy evaluation 是线性方程
对抗训练	内层 max 可能是凸的

启示：理解凸优化 = 理解复杂系统的 “积木块”

Logistic 回归的凸性证明 ★

损失函数：

$$L(w) = \sum_{i=1}^{n} \log(1 + e^{-y_i w^T x_i})$$

要证：$L(w)$ 是凸函数
证明：设 $\ell(z) = \log(1 + e^{-z})$，则 $L(w) = \sum_i \ell(y_i w^T x_i)$
Step 1：证明 $\ell(z)$ 是凸的

$$\ell'(z) = \frac{-e^{-z}}{1 + e^{-z}} = \frac{-1}{1 + e^{z}} = -(1 - \sigma(z))$$

$$\ell''(z) = \sigma(z)(1 - \sigma(z)) \ge 0 \quad \checkmark$$

Step 2：$\ell(y_i w^T x_i)$ 是 w 的凸函数
因为 $y_i w^T x_i$ 是 w 的仿射函数，凸函数复合仿射仍是凸
Step 3：凸函数的和是凸函数 □

加上正则化：$L(w) + \lambda\|w\|_2^2$ 是**强凸**的（$\lambda > 0$）

13 MDP 与线性规划 ★

从实际问题出发

问题 1：库存管理
你经营一家书店，每天要决定进多少货：

- 进多了：库存成本高，书可能卖不掉
- 进少了：缺货损失销售机会
- 需求是随机的（今天可能卖 10 本，也可能卖 50 本）

问题 2：机器维护
你管理一台机器，每天要决定是否维护：

- 机器有 “好”“中”“差” 三种状态
- 不维护：省钱，但机器可能变差
- 维护：花钱，但机器变好
- 机器太差会坏掉，损失更大

问题 3：走迷宫
机器人在网格中找出口：

- 每一步可以上下左右移动
- 有些格子是障碍物
- 目标：用最少步数到达出口

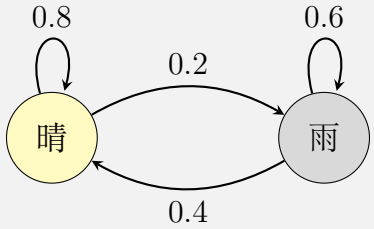
共同特点：状态 → 决策 → 随机转移 → 新状态 → ...

什么是马尔可夫性？

马尔可夫性（Markov Property）：未来只依赖于现在，与过去无关

$$P(s_{t+1}|s_t, s_{t-1}, \ldots, s_0) = P(s_{t+1}|s_t)$$

例子：天气



明天是否下雨，只取决于今天的天气，不取决于昨天、前天...
马尔可夫链：状态按概率转移，没有决策
马尔可夫决策过程 (MDP)：加入决策！你可以选择动作影响转移

MDP 的正式定义

MDP $= (S, A, P, R, \gamma)$

- S：状态空间（有限集合 $\{1, 2, \ldots, n\}$）
- A：动作空间（有限集合）
- $P(s'|s, a)$：转移概率（在状态 s 执行动作 a 后到达 s' 的概率）
- $R(s, a)$：即时奖励（在状态 s 执行动作 a 获得的奖励）
- $\gamma \in [0, 1)$：折扣因子（未来奖励的衰减）

策略：$\pi : S \to A$，告诉你在每个状态该做什么
目标：找最优策略 π^*，最大化累积折扣奖励

$$\mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)\right]$$

为什么要折扣？

- 今天的 1 元比明天的 1 元更值钱
- 数学上保证级数收敛
- γ 接近 1：更有远见；γ 接近 0：更短视

13.1 具体例子：机器维护问题

机器维护 MDP

状态：$S = \{好, 中, 差\}$
动作：$A = \{不维护, 维护\}$
转移概率（不维护时机器会慢慢变差）：

$P(\cdot\|s, 不维护)$	好	中	差
好	0.7	0.3	0
中	0	0.6	0.4
差	0	0	1

$P(\cdot\|s, 维护)$	好	中	差
好	1	0	0
中	0.5	0.5	0
差	0.2	0.6	0.2

奖励（好机器生产多，维护要花钱）：

$R(s, a)$	不维护	维护
好	10	7
中	5	2
差	-1	-4

问题：应该什么时候维护？

机器维护：两种策略对比

策略 1：从不维护

- 短期省钱
- 长期机器变差，损失大

策略 2：状态“中”或“差”时维护

- 维护有成本
- 但机器保持好状态，长期收益高

哪个更好？需要计算！
设 $\gamma = 0.9$，我们来求最优策略...

13.2 Bellman 方程

值函数：衡量状态的好坏

定义 13.1 (状态值函数). 策略 π 下，从状态 s 出发的期望累积奖励：

$$V^\pi(s) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty}\gamma^t R(s_t, \pi(s_t)) \mid s_0 = s\right]$$

直觉：$V^\pi(s)$ 告诉你 "从状态 s 出发，按策略 π 走，期望能拿多少奖励"

最优值函数：

$$V^*(s) = \max_\pi V^\pi(s)$$

这是所有策略中能达到的最大值

Bellman 方程：递归结构

定理 13.2 (Bellman 期望方程).

$$V^\pi(s) = R(s, \pi(s)) + \gamma \sum_{s'} P(s'|s, \pi(s))V^\pi(s')$$

翻译：从 s 出发的总价值 = 立即奖励 + 折扣 × 下一状态的期望价值

定理 13.3 (Bellman 最优方程).

$$V^*(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a)V^*(s')\right]$$

翻译：最优价值 = 选择最好的动作，获得立即奖励 + 折扣 × 下一状态的最优价值

机器维护：计算值函数

设 $\gamma = 0.9$，Bellman 最优方程：

状态 "好"：

$$V^*(\text{好}) = \max\begin{cases} 10 + 0.9(0.7V^*(\text{好}) + 0.3V^*(\text{中})) & \text{不维护} \\ 7 + 0.9 \cdot V^*(\text{好}) & \text{维护} \end{cases}$$

状态 "中"：

$$V^*(\text{中}) = \max\begin{cases} 5 + 0.9(0.6V^*(\text{中}) + 0.4V^*(\text{差})) & \text{不维护} \\ 2 + 0.9(0.5V^*(\text{好}) + 0.5V^*(\text{中})) & \text{维护} \end{cases}$$

状态 "差"：

$$V^*(\text{差}) = \max \begin{cases} -1 + 0.9 \cdot V^*(\text{差}) & \text{不维护} \\ -4 + 0.9(0.2V^*(\text{好}) + 0.6V^*(\text{中}) + 0.2V^*(\text{差})) & \text{维护} \end{cases}$$

这是一个非线性方程组（因为有 max）... 如何求解?

13.3 MDP 的线性规划形式

关键观察：把 max 变成不等式

Bellman 最优方程：

$$V^*(s) = \max_a \left[R(s,a) + \gamma \sum_{s'} P(s'|s,a) V^*(s') \right]$$

等价于：$V^*(s)$ 是满足以下条件的**最小**的 V
对所有状态 s 和动作 a：

$$V(s) \geq R(s,a) + \gamma \sum_{s'} P(s'|s,a) V(s')$$

为什么？

- V^* 显然满足这些不等式（因为 max $\geq$ 任何一项）
- 如果 V 满足所有不等式且 $V < V^*$，则存在某个 s 使得
  $V(s) < V^*(s) = R(s,a^*) + \gamma \sum_{s'} P(s'|s,a^*) V^*(s')$
  但 $V(s) \geq R(s,a^*) + \gamma \sum_{s'} P(s'|s,a^*) V(s') \geq R(s,a^*) + \gamma \sum_{s'} P(s'|s,a^*) V^*(s')$
  矛盾!

MDP 的 LP 形式

定理 13.4 (MDP 的线性规划). *最优值函数 V^* 是以下 LP 的解：*

$$\begin{aligned} \min_V \quad & \sum_s \mu(s) V(s) \\ s.t. \quad & V(s) \geq R(s,a) + \gamma \sum_{s'} P(s'|s,a) V(s'), \quad \forall s \in S, \forall a \in A \end{aligned}$$

其中 $\mu(s) > 0$ 是任意正的权重。

这是一个标准的 LP!

- 变量：$V(s)$，共 $|S|$ 个

- 约束：每对 (s,a) 一个，共 $|S|\times|A|$ 个
- 目标：线性
- 约束：线性

可以用单纯形法或内点法求解！

机器维护问题的 LP

状态：好 (G)、中 (M)、差 (B)，动作：不维护 (N)、维护 (R)

变量：V_G, V_M, V_B

LP（取 $\mu=(1/3,1/3,1/3)$）：

$$\min \frac{1}{3}(V_G+V_M+V_B)$$

约束（6 个，每个 (s,a) 一个）：

$$\begin{aligned}
V_G &\geq 10+0.9(0.7V_G+0.3V_M) && \text{(好, 不维护)}\\
V_G &\geq 7+0.9V_G && \text{(好, 维护)}\\
V_M &\geq 5+0.9(0.6V_M+0.4V_B) && \text{(中, 不维护)}\\
V_M &\geq 2+0.9(0.5V_G+0.5V_M) && \text{(中, 维护)}\\
V_B &\geq -1+0.9V_B && \text{(差, 不维护)}\\
V_B &\geq -4+0.9(0.2V_G+0.6V_M+0.2V_B) && \text{(差, 维护)}
\end{aligned}$$

用 CVXPY 求解

```python
import cvxpy as cp
import numpy as np

# 变量
V = cp.Variable(3)  # V[0]=V_G, V[1]=V_M, V[2]=V_B
gamma = 0.9

# 目标
obj = cp.Minimize(cp.sum(V) / 3)

# 约束
constraints = [
    # 状态 好
```

```
    V[0] >= 10 + gamma * (0.7*V[0] + 0.3*V[1]),  # 不维护
    V[0] >= 7 + gamma * V[0],                     # 维护
    # 状态 中
    V[1] >= 5 + gamma * (0.6*V[1] + 0.4*V[2]),   # 不维护
    V[1] >= 2 + gamma * (0.5*V[0] + 0.5*V[1]),   # 维护
    # 状态 差
    V[2] >= -1 + gamma * V[2],                    # 不维护
    V[2] >= -4 + gamma * (0.2*V[0] + 0.6*V[1] + 0.2*V[2])  # 维
        护
]

prob = cp.Problem(obj, constraints)
prob.solve()
print(f"V* = {V.value}")  # [73.8, 64.5, 47.2] 大约
```

从值函数恢复最优策略

求解 LP 得到：$V_G^* \approx 73.8$，$V_M^* \approx 64.5$，$V_B^* \approx 47.2$

最优策略：选择使 Bellman 方程取等的动作

状态 “好”：

- 不维护：$10 + 0.9(0.7 \times 73.8 + 0.3 \times 64.5) \approx 73.8$ ✓
- 维护：$7 + 0.9 \times 73.8 \approx 73.4$

$\Rightarrow$ **不维护**

状态 “中”：

- 不维护：$5 + 0.9(0.6 \times 64.5 + 0.4 \times 47.2) \approx 56.8$
- 维护：$2 + 0.9(0.5 \times 73.8 + 0.5 \times 64.5) \approx 64.5$ ✓

$\Rightarrow$ **维护**

状态 “差”：

- 不维护：$-1 + 0.9 \times 47.2 \approx 41.5$
- 维护：$-4 + 0.9(0.2 \times 73.8 + 0.6 \times 64.5 + 0.2 \times 47.2) \approx 47.2$ ✓

$\Rightarrow$ **维护**

最优策略：好 $\rightarrow$ 不维护，中 $\rightarrow$ 维护，差 $\rightarrow$ 维护

13.4 LP 对偶与占用度量

MDP 的对偶 LP

原问题有对偶！引入对偶变量 $\rho(s,a) \geq 0$

定理 13.5 (MDP 的对偶 LP).

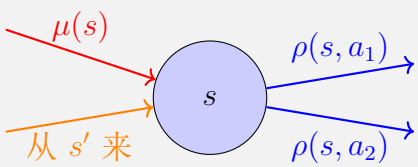
$$\begin{aligned} \max_{\rho} \quad & \sum_{s,a} \rho(s,a) R(s,a) \\ s.t. \quad & \sum_{a} \rho(s,a) = \mu(s) + \gamma \sum_{s',a'} \rho(s',a') P(s|s',a'), \quad \forall s \\ & \rho(s,a) \geq 0 \end{aligned}$$

$\rho(s,a)$ **的含义**：**占用度量**（Occupancy Measure）
$\rho(s,a)$ = 在最优策略下，处于状态 s 并执行动作 a 的（折扣）频率

对偶约束的直觉：流量守恒

约束：$\sum_a \rho(s,a) = \mu(s) + \gamma \sum_{s',a'} \rho(s',a') P(s|s',a')$
翻译：

$$\underbrace{\sum_{a} \rho(s,a)}_{\text{从}s\text{ 流出}} = \underbrace{\mu(s)}_{\text{初始流入}} + \underbrace{\gamma \sum_{s',a'} \rho(s',a') P(s|s',a')}_{\text{从其他状态转移来}}$$



流入 = 流出

这就是网络流的守恒定律！

原问题 vs 对偶问题

	原问题	**对偶问题**
变量	$V(s)$：值函数	$\rho(s,a)$：占用度量
数量	$\lvert S\rvert$ 个	$\lvert S\rvert \times \lvert A\rvert$ 个
约束	Bellman 不等式	流量守恒
目标	最小化总值	最大化总奖励
视角	“状态有多好？”	“应该怎么做？”

强对偶：两个 LP 的最优值相等！（凸优化的对偶理论）

从对偶解恢复策略：

$$\pi^*(a|s) = \frac{\rho^*(s,a)}{\sum_{a'} \rho^*(s,a')}$$

互补松弛在 MDP 中的意义

KKT 互补松弛：$\rho^*(s,a) \cdot [V^*(s) - R(s,a) - \gamma \sum_{s'} P(s'|s,a)V^*(s')] = 0$

翻译：

- 如果 $\rho^*(s,a) > 0$（动作 a 在状态 s 被执行）

  $\Rightarrow$ Bellman 不等式取等（a 是最优动作）

- 如果动作 a 不是最优的（Bellman 不等式严格）

  $\Rightarrow \rho^*(s,a) = 0$（这个动作不会被执行）

经济学解释：

- $V^*(s)$ = 状态 s 的 "影子价格"
- 只有 "物有所值" 的动作才会被使用
- 次优动作的 "使用量" 为零

直觉：MDP-LP 总结

核心结论：

- MDP 的最优策略可以通过 LP 精确求解
- 原问题变量是值函数，对偶变量是占用度量
- LP 对偶揭示了 "评估" 与 "决策" 的深层联系

为什么这很重要？

- 把决策问题转化为我们熟悉的凸优化
- KKT 条件给出最优性的刻画
- 影子价格、互补松弛都有自然解释

局限性：

- 需要知道完整的 MDP 模型（P, R）
- 状态空间大时（如围棋 10^{170} 状态），LP 不可行

- 实际中常用近似方法（值迭代、策略迭代、函数逼近）

但 LP 视角提供了理论基础！

14 GPU 改变现代优化

经典优化 vs 现代优化

经典优化（1960s-2000s）：

- 内点法、单纯形法
- 每步计算量大，但步数少
- 适合中等规模问题（$n < 10^5$）
- 需要精确的二阶信息（Hessian）

现代优化（2010s-）：

- 随机梯度下降（SGD）及其变体
- 每步计算量小，但步数多
- 适合超大规模问题（$n > 10^9$）
- 只需要一阶信息（梯度）

为什么变了？

GPU 视角：GPU 的出现改变了游戏规则

内点法的瓶颈：
每步需要解线性系统 $H\Delta x = -g$，其中 H 是 Hessian

- 形成 H：$O(n^2)$ 存储
- 求解线性系统：$O(n^3)$ 或稀疏情况 $O(n^{1.5})$
- **很难并行化**

梯度下降的优势：
每步只需计算 $x \leftarrow x - \alpha \nabla f(x)$

- 只需 $O(n)$ 存储

- 计算梯度：可以高度并行化
- **完美匹配 GPU 架构**

数值对比

问题：$n = 10^6$ 变量的凸优化

方法	每步复杂度	收敛步数	实际时间
内点法 (CPU)	$O(n^3)$	~ 50	几天（内存不够）
梯度下降 (CPU)	$O(n)$	$\sim 10^4$	几小时
梯度下降 (GPU)	$O(n)$ 并行	$\sim 10^4$	**几分钟**

深度学习的选择：
神经网络参数量：$n \sim 10^9$（GPT-3 有 1750 亿参数）
内点法？绝对不可能。只能用 SGD + GPU。

批量优化：GPU 的超能力

场景：需要解 1000 个独立的小优化问题
传统方法（顺序）：

- 每个问题 0.1 秒
- 总时间：$1000 \times 0.1 = 100$ 秒

GPU 方法（并行）：

- 1000 个问题同时求解
- 总时间：≈ 0.1 秒（几乎和 1 个问题一样）

加速比：$\sim 1000\times$
应用：

- 强化学习：同时模拟 1000 个环境
- 超参数搜索：同时训练 1000 个模型
- 蒙特卡洛：同时采样 1000 条路径

直觉：理论 vs 实践的张力

理论说：
凸优化用内点法，$O(\sqrt{n})$ 步收敛，每步 $O(n^3)$
实践说：
管他收敛多少步，能在 GPU 上跑、能处理 10^9 变量才是王道
调和：

- 小问题（$n < 10^4$）：内点法仍是王者
- 大问题（$n > 10^6$）：一阶方法 + GPU
- 结构化问题：针对性算法（如 ADMM）

本课程：两种视角都要学，知道何时用哪种

15 实验

15.1 实验 2a：CVXPY 求解 SVM 对偶

实验：观察 KKT 条件与支持向量

目标：

- 用 CVXPY 求解 SVM 对偶问题
- 观察互补松弛条件
- 识别支持向量

SVM 原始问题：

$$\min_{w,b} \frac{1}{2}\|w\|^2 \quad \text{s.t.} \quad y_i(w^T x_i + b) \geq 1$$

SVM 对偶问题：

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j x_i^T x_j \quad \text{s.t.} \quad \alpha_i \geq 0, \sum_i \alpha_i y_i = 0$$

实验代码框架

```python
import cvxpy as cp
import numpy as np

```

```python
# 生成数据
np.random.seed(42)
n_samples = 50
X_pos = np.random.randn(n_samples//2, 2) + [2, 2]
X_neg = np.random.randn(n_samples//2, 2) + [-2, -2]
X = np.vstack([X_pos, X_neg])
y = np.array([1]*(n_samples//2) + [-1]*(n_samples//2))

# 对偶问题
n = len(y)
alpha = cp.Variable(n)
K = (X @ X.T) * np.outer(y, y)  # Gram 矩阵

objective = cp.Maximize(cp.sum(alpha) - 0.5 * cp.quad_form(alpha
    , K))
constraints = [alpha >= 0, alpha @ y == 0]
prob = cp.Problem(objective, constraints)
prob.solve()

# 找支持向量
alpha_val = alpha.value
support_vectors = np.where(alpha_val > 1e-5)[0]
print(f"支持向量数量: {len(support_vectors)}")
print(f"支持向量的 alpha: {alpha_val[support_vectors]}")
```

观察互补松弛

KKT 互补松弛：$\alpha_i(1 - y_i(w^T x_i + b)) = 0$

```python
# 从对偶解恢复原始变量
w = sum(alpha_val[i] * y[i] * X[i] for i in range(n))
# 从支持向量计算 b
sv_idx = support_vectors[0]
b = y[sv_idx] - w @ X[sv_idx]

# 验证互补松弛
for i in range(n):
    margin = y[i] * (w @ X[i] + b)
    slack = alpha_val[i] * (1 - margin)
```

```
    if alpha_val[i] > 1e-5:
        print(f"点 {i}: alpha={alpha_val[i]:.4f}, "
              f"margin={margin:.4f}, slack={slack:.6f}")
```

预期结果：

- $\alpha_i > 0$ 的点：margin ≈ 1（在边界上）
- $\alpha_i = 0$ 的点：margin > 1（远离边界）

15.2 实验 2b：GPU vs CPU 凸优化

实验：规模与方法的权衡

目标：比较不同规模下的最优策略

规模	推荐方法	原因
小 ($n \sim 100$)	CVXPY 内点法	精确、快速
中 ($n \sim 10^4$)	视情况	过渡区域
大 ($n \sim 10^5+$)	GPU 梯度下降	内点法内存爆炸

实验代码：小规模 QP

```
import cvxpy as cp
import numpy as np
import time

# 小规模 QP: n = 100
n = 100
np.random.seed(42)
P = np.random.randn(n, n)
P = P.T @ P + 0.1 * np.eye(n)  # 确保正定
q = np.random.randn(n)

x = cp.Variable(n)
objective = cp.Minimize(0.5 * cp.quad_form(x, P) + q @ x)
constraints = [x >= 0]
prob = cp.Problem(objective, constraints)

start = time.time()
```

```python
prob.solve()
print(f"CVXPY (n={n}): {time.time() - start:.4f} 秒")
print(f"最优值: {prob.value:.4f}")
```

预期：内点法在毫秒级完成

实验代码：大规模 QP

```python
import torch

# 大规模 QP: n = 50000
n = 50000
device = 'cuda' if torch.cuda.is_available() else 'cpu'

P = torch.randn(n, n, device=device)
P = P.T @ P + 0.1 * torch.eye(n, device=device)
q = torch.randn(n, device=device)

# 投影梯度下降
x = torch.zeros(n, device=device, requires_grad=True)
optimizer = torch.optim.Adam([x], lr=0.01)

start = time.time()
for i in range(1000):
    loss = 0.5 * x @ P @ x + q @ x
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    with torch.no_grad():
        x.clamp_(min=0)  # 投影到 x >= 0

    if i % 200 == 0:
        print(f"Step {i}: loss = {loss.item():.4f}")

print(f"GPU 梯度下降 (n={n}): {time.time() - start:.2f} 秒")
```

实验代码：批量优化

```python
# 批量优化：同时解 1000 个独立的小 QP
batch_size = 1000
n = 100

# 生成 1000 个不同的问题
P_batch = torch.randn(batch_size, n, n, device=device)
P_batch = P_batch.transpose(-1, -2) @ P_batch
P_batch = P_batch + 0.1 * torch.eye(n, device=device)
q_batch = torch.randn(batch_size, n, device=device)

x_batch = torch.zeros(batch_size, n, device=device,
                      requires_grad=True)
optimizer = torch.optim.Adam([x_batch], lr=0.1)

start = time.time()
for i in range(500):
    # 批量计算所有问题的目标函数
    loss = 0.5 * torch.einsum('bi,bij,bj->b',
                              x_batch, P_batch, x_batch)
    loss = loss + torch.einsum('bi,bi->b', q_batch, x_batch)
    loss = loss.sum()

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    with torch.no_grad():
        x_batch.clamp_(min=0)

print(f"批量优化 1000 个问题: {time.time() - start:.2f} 秒")
```

预期：1000 个问题的时间 $\approx$ 1 个问题的时间

16 本周总结

核心结论：数学定理总结

凸性判定（会证明 ★）：

- 一阶条件：$f(y) \geq f(x) + \nabla f(x)^T(y-x) \Leftrightarrow f$ 凸
- 二阶条件：$\nabla^2 f(x) \succeq 0$ 对所有 $x \Leftrightarrow f$ 凸
- 强凸：$\nabla^2 f(x) \succeq mI$，$m > 0$

对偶理论：

- 弱对偶（总成立）：$d^* \leq p^*$
- 强对偶（Slater 条件）：$d^* = p^*$
- 对偶间隙：$p^* - d^* \geq 0$

KKT 条件（会推导 ★）：

1. 原始可行：$g_i(x^*) \leq 0$，$h_j(x^*) = 0$
2. 对偶可行：$\lambda_i^* \geq 0$
3. 互补松弛：$\lambda_i^* g_i(x^*) = 0$
4. 稳定性：$\nabla_x L = 0$

核心结论：现代视角

凸优化在机器学习中的地位：

- 经典 ML（SVM、Logistic、Lasso）：凸优化是核心
- 深度学习：非凸，但凸优化思想仍然重要
- 子问题：很多非凸问题包含凸子问题

理论 vs 实践：

- 小规模（$n < 10^4$）：内点法，精确解
- 大规模（$n > 10^6$）：一阶方法 + GPU
- 批量问题：GPU 并行，1000 个 $\approx$ 1 个

对偶的三重价值：

1. 提供下界（最优性证书）
2. 影子价格（敏感性分析）
3. 核技巧（SVM、Kernel 方法）

核心结论：带走的核心技能

概念层面：

- 能判断一个优化问题是否是凸的
- 理解 “凸 = 局部即全局” 的意义
- 知道 KKT 条件的四个组成部分

计算层面：

- 能写出简单问题的 Lagrangian
- 能用 KKT 条件手算小问题
- 能用 CVXPY 求解中等规模凸优化

直觉层面：

- 互补松弛 = 稀缺资源才有价格
- 对偶变量 = 约束的边际价值
- 支持向量 = 对偶变量非零的点

17 思考题

思考题

1. 证明：如果 f 和 g 都是凸函数，则 $\max\{f(x), g(x)\}$ 也是凸函数。

2. 为什么凸优化的等式约束只能是仿射的？举一个反例说明非仿射等式约束会破坏凸性。

3. 考虑问题 $\min x^2$ s.t. $x \geq 2$。

(a) 写出 KKT 条件

(b) 求解最优解和最优 Lagrange 乘子

(c) 验证互补松弛条件

4. 在 SVM 中，如果所有训练样本都正确分类且远离边界，会发生什么？这时有多少支持向量?

5. 设计实验：对于 $n = 1000, 5000, 10000, 50000$ 的 QP 问题，分别用 CVXPY 内点法和 GPU 梯度下降求解，画出 "问题规模 vs 求解时间" 的曲线。在什么规模下 GPU 开始占优势?

18 附录：求解 MDP 的三条路 ★

三条路的出发点

MDP 的核心目标是找最优策略 π^*，等价于求最优值函数 V^* 满足 Bellman 最优方程：

$$V^*(s) = \max_a \left[r(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V^*(s') \right]$$

这是一个关于 $V^* \in \mathbb{R}^{|S|}$ 的**非线性不动点方程**（非线性来自 max）。三条路各有不同的解法思路：

	值迭代	策略迭代	强化学习
需要知道 P?	是	是	否
每步计算量	轻	重（解方程组）	轻（采样）
收敛速度	线性（γ 压缩）	有限步必收敛	慢（方差大）
适合规模	小规模	小规模	大规模

第一条路：值迭代（Value Iteration）

核心思路：直接对 Bellman 最优算子 T 反复迭代。

定义 Bellman 最优算子 $T : \mathbb{R}^{|S|} \to \mathbb{R}^{|S|}$：

$$(TV)(s) = \max_a \left[r(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V(s') \right]$$

算法：从任意初始值 $V^{(0)}$（如全零）出发，迭代：

$$V^{(t+1)}(s) = \max_a \left[r(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V^{(t)}(s') \right]$$

收敛性证明：T 是 γ-压缩映射（在 $\|\cdot\|_\infty$ 意义下）。

引理 18.1 (T 是 γ-压缩映射)**.** 对任意 $V, V' \in \mathbb{R}^{|S|}$：

$$\|TV - TV'\|_\infty \le \gamma \|V - V'\|_\infty$$

证明：对任意状态 s，设 $a^* = \arg\max_a [r(s,a) + \gamma \sum_{s'} P(s'|s,a)V(s')]$，

$$\begin{aligned}(TV)(s) - (TV')(s) &\le r(s,a^*) + \gamma \sum_{s'} P(s'|s,a^*)V(s') \\ &\quad - r(s,a^*) - \gamma \sum_{s'} P(s'|s,a^*)V'(s') \\ &= \gamma \sum_{s'} P(s'|s,a^*)\big[V(s') - V'(s')\big] \\ &\le \gamma \sum_{s'} P(s'|s,a^*)\|V - V'\|_\infty \\ &= \gamma \|V - V'\|_\infty\end{aligned}$$

对称地，$(TV')(s) - (TV)(s) \le \gamma\|V - V'\|_\infty$，故 $|(TV)(s) - (TV')(s)| \le \gamma\|V - V'\|_\infty$，对所有 s 取上确界得结论。□

由 Banach 不动点定理，T 有唯一不动点 $V^* = TV^*$，且从任意初始点出发：

$$\|V^{(t)} - V^*\|_\infty \le \gamma^t \|V^{(0)} - V^*\|_\infty$$

误差每步缩小 γ 倍，**线性收敛**，步数 $O\left(\frac{\log(1/\varepsilon)}{1-\gamma}\right)$。

每步代价：遍历所有 (s,a) 对，计算量 $O(|S|^2|A|)$（含转移概率求和）。

缺点：状态空间爆炸（Atari 游戏的像素空间 $\sim 10^{100000}$）完全无法枚举。

第二条路：策略迭代（Policy Iteration）

核心思路：交替做「策略评估」和「策略改进」。

算法框架：从任意初始策略 $\pi^{(0)}$ 出发，交替执行：

步骤 1——策略评估（Policy Evaluation）：固定策略 π，求对应的值函数 V^π。

V^π 满足线性 Bellman 方程：

$$V^\pi(s) = r(s,\pi(s)) + \gamma \sum_{s'} P(s'|s,\pi(s))\, V^\pi(s')$$

写成矩阵形式：设 $[P_\pi]_{ss'} = P(s'|s,\pi(s))$，$r_\pi(s) = r(s,\pi(s))$，

$$V^\pi = r_\pi + \gamma P_\pi V^\pi \implies \underbrace{(I - \gamma P_\pi)}_{\text{可逆，因 } \gamma < 1} V^\pi = r_\pi \implies V^\pi = (I - \gamma P_\pi)^{-1} r_\pi$$

$(I - \gamma P_\pi)$ 可逆的原因：$\gamma < 1$ 使其严格对角占优。可以直接求逆 $O(|S|^3)$ 或用迭代法（Gauss-Seidel）。

步骤 2——策略改进（Policy Improvement）：用 V^π 贪心更新策略：

$$\pi'(s) = \arg\max_a \left[r(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V^\pi(s') \right]$$

定理 18.2 (策略改进定理). $V^{\pi'} \geq V^{\pi}$（逐分量不等式），且若 $\pi' = \pi$，则 π 已是最优策略。

证明（单步不等式）：

$$
\begin{aligned}
V^{\pi}(s) &= r(s,\pi(s)) + \gamma \sum_{s'} P(s'|s,\pi(s)) V^{\pi}(s') \\
&\leq \max_a \left[r(s,a) + \gamma \sum_{s'} P(s'|s,a) V^{\pi}(s') \right] \\
&= r(s,\pi'(s)) + \gamma \sum_{s'} P(s'|s,\pi'(s)) V^{\pi}(s') \\
&\leq r(s,\pi'(s)) + \gamma \sum_{s'} P(s'|s,\pi'(s)) V^{\pi'}(s') \\
&= V^{\pi'}(s) \qquad \square
\end{aligned}
$$

（第二个不等号递归展开，需用 $V^{\pi} \leq V^{\pi'}$ 归纳。）
收敛性：策略空间有限（最多 $|A|^{|S|}$ 个确定性策略），每次迭代严格改进（$V^{\pi'} > V^{\pi}$ 除非已最优），故**有限步内必收敛**，与 γ 无关。
与值迭代的比较：策略迭代每步计算量大（$O(|S|^3)$ 解线性方程组），但步数通常远少于值迭代——实践中几步到几十步即可收敛。

第三条路：强化学习（不知道 P）

前两条路都假设已知转移概率 $P(s'|s,a)$（model-based）。现实中 P 往往未知，只能通过与环境交互采样得到 (s,a,r,s')。
Q-learning（值函数法，off-policy）
维护动作值函数 $Q(s,a)$ 的估计，每次观测 (s,a,r,s') 更新：

$$
Q(s,a) \;\leftarrow\; Q(s,a) + \alpha \underbrace{\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]}_{\text{TD 误差（Temporal Difference Error）}}
$$

- 括号里是**时序差分误差**（TD error）：当前估计与 Bellman 方程右端之差
- α 是学习率（步长），类比随机梯度下降
- 无需知道 P，直接用样本 s' 近似期望
- 收敛性：在一定条件下（每个 (s,a) 访问无穷次，学习率满足 Robbins-Monro 条件）收敛到 Q^*

策略梯度（Policy Gradient，直接优化策略参数）

参数化策略 $\pi_\theta(a|s)$（如神经网络），直接对参数 θ 做梯度上升：

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a|s) \cdot Q^{\pi_\theta}(s,a)\right]$$

- 右侧期望用轨迹采样估计（Monte Carlo 或 TD）
- $\nabla_\theta \log \pi_\theta(a|s)$：对数策略梯度（score function），指向增加动作 a 概率的方向
- $Q^{\pi_\theta}(s,a)$：该动作的”权重”，好动作权重高，坏动作权重低
- **与 LP 对偶的联系**：策略梯度在**占用度量**（occupancy measure）空间里做的，正是 LP 对偶问题的梯度上升——两者的对象是同一个优化问题的两个侧面

与 LP 框架的根本区别

方法	本质	优缺点
值迭代 / 策略迭代	不动点迭代（函数空间）	有理论保证，难以加约束
LP 方法	全局优化问题	有对偶结构，可加约束，但规模受限
强化学习	随机近似 / 梯度估计	可扩展，但收敛慢、方差大

核心结论：为什么安全强化学习要回到 LP？

值迭代和策略迭代是**不动点迭代**，本质在函数空间里找压缩映射的极限，**加约束极为麻烦**（约束会破坏压缩映射结构）。
LP 把 Bellman 不动点转化为**全局优化问题**，约束直接加到 LP 里——这正是**约束马尔可夫决策过程**（Constrained MDP, CMDP）和**安全强化学习**最终要借助 LP 或拉格朗日对偶框架的根本原因。